

TECHNICAL SPECIFICATION

Fitness Chatbot Ollama Cloud Integration

How the exercise and meal databases are consumed by a model hosted on Ollama Cloud, and how the Flutter app, the gateway and the Ollama Cloud API fit together.

Version 1.1 · **Date** 25 July 2026

Serving Ollama Cloud · **Model** `gemma4:31b`

Schema version 1 · **Data** 2026.07.25

Audience Implementation team

Scope exercises.db · meals.db · gateway · Flutter chat · deep links

Contents

1. Overview	5
1.1 What this document covers	5
1.2 Why a gateway exists at all	5
1.3 The two copies of the data	6
1.4 Request lifecycle	6
1.5 Latency budget	7
1.6 What Ollama Cloud does and does not give you	8
2. The databases	9
2.1 Files, versioning and the build	9
2.2 Design decisions that matter downstream	9
2.3 exercises.db — tables	10
2.4 meals.db — tables	11
2.5 The card views — the retrieval contract	11
2.6 Full-text search	12
3. How the model uses the database	13
3.1 The principle	13
3.2 Tier 1 — the vocabulary block	13
3.3 The system prompt	14
3.4 Tool definitions	15
3.5 Compiling a tool call to SQL	15
3.6 The relaxation ladder	16
3.7 Serializing candidates for the prompt	17
3.8 Turn B — schema-constrained output	17
3.9 Validating what comes back	18
3.10 Worked example	18
3.11 Context budget	19
3.12 Ollama Cloud request configuration	20
3.13 Model choice — gemma4:31b	21
3.14 The cost model: GPU-time, not tokens	22
4. The gateway	23
4.1 Responsibilities	23
4.2 Endpoints	23
4.3 The SSE protocol	24
4.4 Streaming the reply field without parsing partial JSON	25
4.5 The two-turn loop	26

4.6 The upstream client — key, concurrency, retries	28
4.7 The fast path	30
4.8 Auth, rate limiting and quota	31
4.9 Logging	32
4.10 Error handling	33
4.11 Degraded mode — retrieval without the model	33
5. The Flutter app	35
5.1 Packages	35
5.2 Installing the databases on first run	35
5.3 The data access layer	36
5.4 The card model	38
5.5 Consuming the SSE stream	39
5.6 Wiring events to a message	41
5.7 Rendering, and the tap	42
5.8 Offline and degraded behaviour	43
6. Deep linking	44
6.1 One route table	44
6.2 Handling the action field	44
6.3 Why ids, and only ids	45
6.4 The stale-data path	46
6.5 Analytics closing the loop	46
7. Build, deploy and operate	47
7.1 The data pipeline	47
7.2 Deploying a new data version	47
7.3 Connecting to Ollama Cloud	47
7.4 Where to run the gateway	49
7.5 Evaluation set	50
7.6 What to watch in production	50
7.7 Risks	51
7.8 Implementation order	52
Appendix A — Full DDL	54
A.1 exercises.db	54
A.2 meals.db	55
Appendix B — SQL cookbook	57
B.1 Retrieval — the query behind search_exercises	57
B.2 Retrieval — the query behind search_meals	57
B.3 Bilingual full-text search	57
B.4 Hydrating validated ids into cards	58

B.5 Everything that trains a muscle, in any role	58
B.6 Cross-database query	58
B.7 Building the vocabulary block	58
B.8 Operational checks	58
B.9 Query plans to sanity-check	59
Appendix C — Reference	60
C.1 Files at a glance	60
C.2 Validation report — error kinds	60
C.3 Validation report — warning kinds	61
C.4 Vocabulary counts (current data)	61
C.5 Glossary	61
C.6 Pre-launch checklist	62

1. Overview

1.1 What this document covers

How the exercise and meal databases built by `build_catalog_db.py` are consumed by a language model hosted on Ollama Cloud, and how the Flutter app, a gateway service and the Ollama Cloud API fit together to produce a chatbot that recommends **real catalog items** and deep-links to the screens that already exist in the app.

Three components, three responsibilities:

Component	Owns	Never does
Flutter app	Rendering, navigation, the offline catalog, the local copy of the databases	Talk to Ollama Cloud directly; hold the API key; construct prompts
Gateway (your server)	The system prompt, tool definitions, retrieval SQL, ref validation, auth, rate limiting, the API key, and the upstream quota	Ship anything to the app that it hasn't validated
Ollama Cloud	Language understanding and generation only	See the database, your network, or user identity

The single most important design rule in this document:

The model never sees the database and never invents an identifier. It emits *tool calls* describing what it wants; the gateway compiles those into parameterised SQL; the rows that come back are the only exercises and meals the model is permitted to cite.

Everything else — streaming, deep links, offline behaviour, error handling — follows from that rule. None of it depends on where the model runs, because retrieval was never the model's job.

1.2 Why a gateway exists at all

It is tempting to have the Flutter app call Ollama Cloud directly. Don't. The thing you would be shipping into the app is **a credential attached to your account and your bill**.

- **The API key cannot live in the app.** A key shipped in a binary is a key in the hands of anyone who unzips the binary. It is not obfuscatable and not revocable per user — you would be rotating a single key against the entire installed base. The key belongs in the gateway's secret store and nowhere else.
- **Ollama Cloud has no concept of your users.** It authenticates your account, not your customers. Per-user rate limits, quotas and bans can only exist on a server you control.
- **Quota is a shared, exhaustible resource.** One abusive client can spend the allowance that every other user depends on. Only the gateway can meter that.
- **Prompts change constantly.** The system prompt is the thing you will iterate on most. On the server it is a config file; in the app it is an App Store review.

- **Retrieval needs to be deterministic and auditable.** Compiling tool calls to SQL on the server means you can log every candidate set and diagnose bad answers later.
- **Validation must not be client-side.** If the app decided which ids to trust, a hallucinated id would navigate the user to a wrong or missing screen.

The app keeps its own copy of the databases anyway — but for **rendering and offline browsing**, not for prompting.

1.3 The two copies of the data

One build pipeline, two destinations:



Both are stamped with the same `data_version`. The gateway returns its version with every response; when the app's copy is older it still renders, using the denormalized snapshot the gateway attaches to each reference (see §6.4).

Note that the gateway's copy of the databases is now the *only* stateful thing on your infrastructure. There is no model to host, no weights to store, no GPU to keep warm — the gateway is a stateless HTTP service with two small read-only SQLite files next to it. That is a materially cheaper thing to run and to scale horizontally.

1.4 Request lifecycle

```

user types "give me some beginner glute exercises, I only have a mat"
|
v
[Flutter] POST /v1/chat {message, locale, profile, conversation_id}
|
v
[Gateway] build messages: system prompt + vocabulary + history + user
|
|--> [Ollama Cloud] POST https://ollama.com/api/chat
|      Authorization: Bearer $OLLAMA_API_KEY
|      model=gemma4:31b think=false tools=[search_exercises,
|                                     search_meals, search_by_text]
|      <-- tool_calls: search_exercises{muscle_group:"Glutes",
|                                     equipment:"Bodyweight",
|                                     max_difficulty:2, limit:6}
|
|--> [SQLite] compile to parameterised SQL against exercises.db -> 2 rows
|      CANDIDATE SET = {bodyweight-glute-bridge, bodyweight-hip-thrust, ...}
|
|--> [Ollama Cloud] POST /api/chat (tool result appended,
|      format=<JSON schema>, stream=true)
|      <-- {"reply": "...", "exercise_refs":["bodyweight-glute-bridge", ...]}
  
```

```
|-- validate: refs n CANDIDATE SET -> drop anything invented
v
[Flutter] SSE: event:token xN -> event:refs -> event:done
|
v
ExerciseCard carousel, hydrated from the LOCAL exercises.db
|
|-- tap --> go_router /exercises/bodyweight-glute-bridge
```

Two Ollama Cloud round trips per user message. §4.7 covers the fast path that skips the first one when the request is unambiguous, removing one of the two upstream requests and the prefill that goes with it. That is a cost lever as much as a latency one.

1.5 Latency budget

The budget is dominated by two things you do not control: the network path from your gateway to `ollama.com`, and whatever queueing your plan's concurrency allowance imposes.

Stage	Budget	Notes
App → gateway	30–80 ms	Depends on the user's network
Gateway → ollama.com RTT	see §7.4	The new term. Low if the gateway is in a US region; 200 ms+ round trip if it sits in the Gulf or North Africa. Paid at least twice per message
Turn A (tool call)	0.4–0.9 s	Short output — just the call. Includes one RTT
SQL retrieval	1–4 ms	Indexed; effectively free. Runs on your host
Turn B time-to-first-token	0.5–1.2 s	The number the user actually feels. Includes one RTT
Turn B full generation	1.5–3 s	~180 words. A hosted 31B on datacentre hardware generates faster than a local quantized 7B on a mid-range GPU — expect this to improve, not regress
Total to first token	~1.0–2.2 s	Two-turn path. The fast path (§4.7) removes roughly half

Treat every number above as a **budget to verify, not a measurement**. Nobody has run them against your gateway, in your region, on your plan. Instrument `ttft_ms` from day one (§4.9) and replace this table with your own p50/p95 before anyone quotes it in a planning meeting.

What you don't have to think about. Model residency. There is no cold load to avoid, no `keep_alive` to set — the parameter is not supported on cloud models — and no warm-up call at deploy time.

What you do. Three sources of variance, in rough order of how often they will bite:

- **Reasoning tokens.** gemma4 supports thinking modes. Left on, the model spends real seconds and real GPU-time producing a trace nobody in this system reads. Send `"think": false` on both turns (§3.12).

- **Upstream queueing.** On the Free plan you have one concurrent cloud model. The second simultaneous user waits for the first to finish generating — that is 2–3 seconds of pure queue, not 300 ms. Size the semaphore in §4.6 to your plan and return a busy state rather than letting the queue grow.
- **Geography.** Two round trips per message means the gateway's distance to `ollama.com` is paid twice. §7.4 is the section to read before you pick a region.

1.6 What Ollama Cloud does and does not give you

Property	Where it stands
API surface	The same <code>/api/chat</code> the local daemon serves. Tools, streaming and <code>format</code> (JSON Schema structured output) all behave as documented for local models
Unsupported parameters	<code>num_ctx</code> and <code>keep_alive</code> . Sending them is not an error, but they do nothing — remove them so your logs don't lie to you
Model naming	<code>gemma4:31b</code> when calling <code>ollama.com</code> directly. The <code>-cloud</code> suffix (<code>gemma4:31b-cloud</code>) is for routing <i>through</i> a local Ollama daemon acting as a proxy
Metering	GPU-time, not a fixed token or request count. Model weight class, generation length and prompt length all move the meter
Free plan	One concurrent cloud model and a light allowance, documented as covering lighter model weight classes. Confirm on your own account that <code>gemma4:31b</code> is reachable before planning around it — if it is not, phase 0a (§7.8) is where you find out, and a paid tier moves from "before launch" to "before development"
Data handling	Ollama states prompt and response data is never logged or trained on. Models are hosted primarily in the United States, with possible routing to Europe and Singapore
Quota visibility	Not exposed via the API. There is no usage header or endpoint to read your remaining allowance from — you cannot query it, so you must estimate it yourself (§3.14) and detect exhaustion reactively from a 429. The exact reset cadence is not documented either; do not build logic that assumes a specific window length
SLA	None you can point at. Design the degraded path in §4.11 as though the model will be unavailable, because at some point it will be

One decision this document does not make for you. The Free plan allows a single concurrent cloud model and a light usage allowance measured in GPU-time. That is enough to build against and to evaluate against; it is not enough to serve concurrent users. Everything below is written so that moving to a paid plan is a configuration change — raise a semaphore, raise a quota — and not a rewrite. Budget for that change before launch, and confirm during phase 0a (§7.8) that `gemma4:31b` is reachable on whatever plan you are actually on.

2. The databases

2.1 Files, versioning and the build

`build_catalog_db.py` reads the JSON assets and writes two SQLite databases:

```
python3 tools/build_catalog_db.py          # all paths default to the project layout
```

File	Contents	Approx. size
<code>exercises.db</code>	exercises, all vocabulary tables, <code>exercise_fts</code> , <code>exercise_card</code>	3–5 MB at ~3,000 rows
<code>meals.db</code>	meals, ingredients, <code>meal_fts</code> , <code>meal_card</code>	1–2 MB at ~300 rows
<code>manifest.json</code>	<code>data_version</code> , SHA-256 and byte size per database	< 2 KB
<code>vocab/*.json</code>	the six controlled-vocabulary files	~40 KB
<code>validation_report.json</code>	every error and warning found during the build	varies

Two databases rather than one so meals can be re-scraped and reshipped without touching the exercise data. Each carries its own `meta` table:

```
SELECT key, value FROM meta;
-- domain          | exercises
-- data_version    | 2026.07.25
-- schema_version  | 1
-- count_exercises | 3012
```

When a query genuinely needs both domains — only the chatbot's retrieval step does — attach them:

```
ATTACH 'exercises.db' AS ex;
ATTACH 'meals.db'     AS ml;

SELECT 'exercise' AS kind, id, name FROM ex.exercise_card
WHERE muscle_group = :group AND difficulty_rank <= 2
UNION ALL
SELECT 'meal', id, name FROM ml.meal_card
WHERE protein_g >= :protein
ORDER BY kind, name;
```

2.2 Design decisions that matter downstream

Exercise ids are slugs derived from the name. The source data has no id field at all.

`slugify("Bodyweight Glute Bridge")` → `bodyweight-glute-bridge`. This is the only identifier that ever leaves the system: the model emits it, `go_router` routes on it, analytics key on it. Renaming an exercise changes its id, so treat renames as a migration.

Muscles are keyed by name, never by the source id. In `prime_mover_muscles.json`, 26 rows share only 17 ids — `58b9bade-...` alone is used for Anterior, Posterior and Medial Deltoids, Infraspinatus,

Subscapularis and Serratus Anterior. Keying on it merges six muscles into one. The original ids survive in `muscle.legacy_ids` and in `legacy_muscle_alias`, where `ambiguous = 1` marks the ones that cannot be reverse-resolved.

Difficulty has a canonical rank. The source file's array order is not the progression — it lists Intermediate before Novice. `difficulty_level.rank` runs 1 (Beginner) to 8 (Legendary), and `exercise.difficulty_rank` is denormalized onto the exercise row so the hot retrieval filter hits a covering index without a join.

Every "no value" spelling becomes SQL `NULL`. The source uses `"", "- ", "N/A", "Unsorted"` and `null` interchangeably. Write `WHERE x IS NULL`, never `WHERE x = ''`.

2.3 exercises.db — tables

`exercise` — one row per exercise.

Column	Type	Notes
<code>id</code>	TEXT PK	Slug. The only public identifier.
<code>name</code> , <code>name_ar</code>	TEXT	<code>name_ar</code> may be NULL until the translation pass runs
<code>difficulty_id</code>	TEXT FK	→ <code>difficulty_level.id</code>
<code>difficulty_rank</code>	INTEGER	1–8, denormalized for the covering index
<code>muscle_group_id</code>	TEXT FK	→ <code>muscle_group.id</code>
<code>prime_mover_id</code>	TEXT FK	→ <code>muscle.id</code> ; also present in <code>exercise_muscle</code> as <code>role='prime'</code>
<code>primary_equipment_id</code> , <code>secondary_equipment_id</code>	TEXT FK	→ <code>equipment.id</code>
<code>body_region_id</code>	TEXT FK	Upper Body / Lower Body / Midsection / Full Body
<code>posture</code> , <code>grip</code> , <code>laterality</code> , <code>mechanics</code> , <code>force_type</code>	TEXT	Free vocabulary, NULL-able
<code>arms_mode</code> , <code>legs_mode</code>	TEXT	Renamed from <code>continuous_or_alternating_*</code>
<code>demo_url</code> , <code>explain_url</code>	TEXT	YouTube links. Never sent to the model — the app alone attaches them at render time
<code>popularity</code>	INTEGER	Starts at 0; drive it from tap analytics

Indexes: `ix_ex_group`, `ix_ex equip`, `ix_ex_diff`, `ix_ex_region`, `ix_ex_mech`, and the covering `ix_ex_filter` (`muscle_group_id`, `primary_equipment_id`, `difficulty_rank`).

`exercise_muscle` — (`exercise_id`, `muscle_id`, `role`) where `role` is `prime | secondary | tertiary`. This is what you query for "everything that trains this muscle in any role", which the `prime_mover_id` column alone cannot answer.

`exercise_movement_pattern`, `exercise_plane` — ordered many-to-many, `position` 1–3, mapping the source's `movement_pattern_1..3` and `plane_of_motion_1..3`.

Vocabulary: `difficulty_level` (8), `muscle_group` (16 after merging), `muscle` (24 after merging), `equipment`, `movement_pattern`, `body_region`, `legacy_muscle_alias`.

2.4 meals.db — tables

`meal` — one row per meal, `id` is TheMealDB's `idMeal`, `slug` is the unique human-readable key.

Column	Type	Notes
<code>kcal</code> , <code>protein_g</code> , <code>carbs_g</code> , <code>fat_g</code>	REAL	Per serving, and estimated. NULL = unknown
<code>nutrition_is_estimate</code>	INTEGER	1 by default. Always check before showing a number
<code>nutrition_confidence</code>	REAL	0.0–1.0 — share of ingredients matched to the nutrition table
<code>servings_assumed</code>	INTEGER	4 unless the scraper was told otherwise
<code>vegetarian</code> , <code>vegan</code> , <code>pescatarian</code>	INTEGER	0/1/NULL, derived from ingredients + category
<code>contains_gluten</code> , <code>contains_dairy_or_egg</code> , <code>contains_nuts</code>	INTEGER	Filtering hints, not allergen guarantees

`meal_ingredient` — (`meal_id`, position) with `qty`, `unit` (normalized through an alias table: `tbs` → `tbsp`, `tin` → `can`) and `measure_raw` (the original string, e.g. "2 1/2 cups"). Display `measure_raw`; compute on `qty / unit`; expect NULLs where the source was unparseable ("sprinkling").

`meal_step`, `meal_tag`, `ingredient`, `meal_category`, `meal_area` complete the set.

2.5 The card views — the retrieval contract

`exercise_card` and `meal_card` are the only things the retrieval layer and the app's list screens read. Both sides use the same view, so the shape of a "card" cannot drift between the chatbot and the rest of the app.

```
SELECT id, name, name_ar, muscle_group, prime_mover, difficulty, difficulty_rank,
       equipment, body_region, mechanics, movement_patterns
FROM exercise_card
WHERE muscle_group = 'Glutes' AND equipment = 'Bodyweight'
LIMIT 3;
```

<code>id</code>	<code>name</code>	<code>muscle_group</code>	<code>prime_mover</code>	<code>difficulty</code>
bodyweight-glute-bridge	Bodyweight Glute Bridge	Glutes	Gluteus Maximus	Beginner
bodyweight-hip-thrust	Bodyweight Hip Thrust	Glutes	Gluteus Maximus	Novice

`movement_patterns` is a comma-joined string built by a correlated subquery, which is why filtering on it uses `LIKE` rather than `=`.

2.6 Full-text search

Both databases carry an FTS5 table tokenized with `unicode61 remove_diacritics 2`, so Arabic and English match from one query and `glute*` prefix search works.

```
SELECT c.id, c.name, c.muscle_group, c.difficulty
  FROM exercise_fts f
 JOIN exercise_card c ON c.id = f.id
 WHERE exercise_fts MATCH :q
 ORDER BY bm25(exercise_fts), c.popularity DESC
 LIMIT :limit;
```

Never pass raw user text into `MATCH`. FTS5 has its own query syntax; an unbalanced quote raises `sqlite3.OperationalError` and `NEAR / OR / column` filters let a user steer the search. Quote every token:

```
import re

_FTS_UNSAFE = re.compile(r'["\'()*:~^~\']')

def fts_escape(q: str) -> str:
    """Wrap each token in double quotes so user text can't inject FTS syntax."""
    tokens = [t for t in _FTS_UNSAFE.sub(" ", q).split() if t]
    return " ".join(f'"{t}"' for t in tokens)

# fts_escape('glute OR name:* --') -> '"glute" "OR" "name"'
```

The FTS tables are **not** kept in sync by triggers — the databases are regenerated, not mutated. If you ever add write paths, add the triggers.

3. How the model uses the database

3.1 The principle

The model has no database driver, no network access and no knowledge of your catalog. What it has is:

1. **A vocabulary** — the legal values for every filter, ~500 tokens in the system prompt, generated from the database at boot.
2. **Three tools** — `search_exercises`, `search_meals`, `search_by_text`, whose arguments are exactly those vocabulary values.
3. **A candidate set** — the ≤ 8 rows the gateway retrieved for *this turn*, injected as a tool result.

It may only cite ids from (3). Everything else is enforced by the gateway.

Why not put the catalog in the prompt?

At roughly 120 tokens per exercise, 3,000 exercises is **~360,000 tokens**. That exceeds even `gemma4:31b`'s 256K window, and the quality would collapse long before the limit. Capacity is only the first objection. The harder one is cost: **you would be paying GPU-time to re-read the entire catalog on every turn**, forever, to answer a question that a 4 ms indexed SQL query answers exactly. The vocabulary is 500 tokens and the candidates are 700 — a 300× reduction that also makes the model's output verifiable.

Why not embeddings?

Structured facets (muscle group, equipment, difficulty, movement pattern) are exactly what fitness questions filter on, and SQL answers them exactly, instantly and explainably. Embeddings would add an on-device model, ~10 MB of vectors and fuzzy results for a problem that has a precise answer. The one thing they'd add — matching phrasings the vocabulary doesn't cover — is handled by `search_by_text` over FTS5.

3.2 Tier 1 — the vocabulary block

Built once at gateway boot, straight from the database, so it can never drift from what retrieval will actually accept:

```
VOCAB_SQL = """
SELECT (SELECT group_concat(name, ' | ') FROM muscle_group      ORDER BY name) AS muscle_groups,
       (SELECT group_concat(name, ' | ') FROM equipment        ORDER BY name) AS equipment,
       (SELECT group_concat(name, ' | ') FROM movement_pattern ORDER BY name) AS patterns,
       (SELECT group_concat(name || '=' || rank, ' | ')
        FROM difficulty_level ORDER BY rank)
                                AS difficulty
""""

def build_vocabulary_block(con) -> str:
    r = con.execute(VOCAB_SQL).fetchone()
    return (
        "CATALOG VOCABULARY – use these exact values in tool arguments:\n"
        f"muscle_group: {r['muscle_groups']}\n"
        f"equipment: {r['equipment']}\n"
        f"movement_pattern: {r['patterns']}\n"
        f"max_difficulty (integer): {r['difficulty']}\n"
        "body_region: Upper Body | Lower Body | Midsection | Full Body\n"
        "mechanics: Compound | Isolation"
    )
```

Rendered:

```
CATALOG VOCABULARY – use these exact values in tool arguments:
muscle_group: Abdominals | Abductors | Adductors | Back | Biceps | Calves | Chest |
Forearms | Glutes | Hamstrings | Hip Flexors | Quadriceps | Shins | Shoulders |
Trapezius | Triceps
equipment: Barbell | Bodyweight | Cable | Dumbbell | Stability Ball
movement_pattern: Anti-Extension | Anti-Rotational | Hip Extension | Hip Hinge |
Horizontal Push | Rotational | Squat | Vertical Pull
max_difficulty (integer): Beginner=1 | Novice=2 | Intermediate=3 | Advanced=4 |
Expert=5 | Master=6 | Grand Master=7 | Legendary=8
body_region: Upper Body | Lower Body | Midsection | Full Body
mechanics: Compound | Isolation
```

Cache it in memory and invalidate on `data_version` change. Do not rebuild it per request.

3.3 The system prompt

Lives on the gateway as a versioned file, never in the app.

You are the in-app fitness coach for {APP_NAME}. You help with training, technique, programming, recovery and nutrition.

LANGUAGE

Reply in the same language the user wrote in. The user's app locale is {locale}. For Arabic, use Modern Standard Arabic and give the English name of each exercise in parentheses on first mention.

USER PROFILE

level={level} | goal={goal} | available equipment={equipment}
injuries/limits={limits} | age_band={age_band} | units={units}

GROUND RULES

1. Before recommending any exercise or meal, call `search_exercises` / `search_meals`. Recommend ONLY items present in the CANDIDATE block returned by those tools. Copy their ids verbatim into `exercise_refs` / `meal_refs`. Never invent an id or an exercise name.
2. If the candidates don't fit, call the tool again with different filters rather than answering from memory.
3. Never output a URL, image, video link or markdown. The app renders those.

4. Respect stated injuries and available equipment absolutely. If the user says they have no equipment, pass equipment="Bodyweight".
5. Meal macros are ESTIMATES. Say "roughly" or "about". If a value is missing, say you don't have it – never guess a number.
6. You are not a doctor. For pain, injury, medication, pregnancy, eating disorders or extreme calorie targets: give general safety guidance, set safety_flag, and recommend a qualified professional. Do not diagnose.
7. Stay on topic: training, nutrition, recovery and this app. Politely redirect anything else and set safety_flag="off_topic".
8. Keep replies under 180 words unless the user asks for a full program.

{VOCABULARY_BLOCK}

3.4 Tool definitions

Passed as the `tools` array on turn A. Ollama Cloud exposes the same OpenAI-style function calling the local daemon does; `gemma4` is trained for it.

```
{
  "type": "function",
  "function": {
    "name": "search_exercises",
    "description": "Find exercises in the app's catalog. Use the exact vocabulary values listed in the system prompt. Call this before recommending any exercise.",
    "parameters": {
      "type": "object",
      "properties": {
        "muscle_group": {"type": "string"},
        "equipment": {"type": "string"},
        "max_difficulty": {"type": "integer", "minimum": 1, "maximum": 8,
          "description": "1=Beginner .. 8=Legendary. Returns everything at or below this level."},
        "movement_pattern": {"type": "string"},
        "body_region": {"type": "string",
          "enum": ["Upper Body", "Lower Body", "Midsection", "Full Body"]},
        "mechanics": {"type": "string", "enum": ["Compound", "Isolation"]},
        "exclude_equipment": {"type": "string"},
        "limit": {"type": "integer", "default": 6, "maximum": 8}
      },
      "required": []
    }
  }
}
```

`search_meals` mirrors it with `category`, `area`, `min_protein`, `max_kcal`, `vegetarian`, `vegan`, `gluten_free`, `exclude_ingredient`. `search_by_text` takes `{query, domain}` and routes to FTS5.

Every field is optional. A model that calls `search_exercises({})` gets the most popular exercises rather than an error — degrade, never fail.

3.5 Compiling a tool call to SQL

The tool arguments map one-to-one onto a single parameterised statement. Missing arguments arrive as `NULL` and disable their own clause, so one query serves every combination of filters.

```
SEARCH_EXERCISES_SQL = ""
SELECT id, name, name_ar, muscle_group, prime_mover, difficulty, difficulty_rank,
       equipment, mechanics, body_region, movement_patterns
```

```

FROM exercise_card
WHERE (:muscle_group IS NULL OR muscle_group = :muscle_group)
      AND (:equipment IS NULL OR equipment = :equipment)
      AND (:body_region IS NULL OR body_region = :body_region)
      AND (:mechanics IS NULL OR mechanics = :mechanics)
      AND (:max_difficulty IS NULL OR difficulty_rank <= :max_difficulty)
      AND (:movement_pattern IS NULL OR movement_patterns LIKE '%' || :movement_pattern || '%')
      AND (:exclude_equipment IS NULL OR equipment IS NOT :exclude_equipment)
ORDER BY popularity DESC, difficulty_rank ASC, name ASC
LIMIT :limit
"""

DEFAULTS = {"muscle_group": None, "equipment": None, "body_region": None,
            "mechanics": None, "max_difficulty": None, "movement_pattern": None,
            "exclude_equipment": None, "limit": 8}

```

`equipment IS NOT :exclude_equipment` rather than `<>` — `<>` is NULL-propagating, so rows whose equipment is NULL would silently vanish whenever the user excludes something.

Never string-format a tool argument into SQL. The arguments come from a language model that can be steered by the user; treat them as untrusted input — doubly so given that the model runs on infrastructure you don't control.

3.6 The relaxation ladder

Zero results is the failure mode that makes a chatbot look broken. Rather than returning nothing, drop filters in a fixed order — least to most semantically important — and tell the model what was dropped so it can say so.

```

FILTER_ORDER = ["movement_pattern", "mechanics", "body_region",
                "max_difficulty", "equipment", "muscle_group"]

def search_exercises(con, **facets):
    params = {**DEFAULTS, **facets}
    relaxed = []
    for _ in range(len(FILTER_ORDER) + 1):
        rows = con.execute(SEARCH_EXERCISES_SQL, params).fetchall()
        if rows:
            return rows, relaxed
        for key in FILTER_ORDER:
            if params.get(key) is not None:
                params[key] = None
                relaxed.append(key)
                break
        else:
            break
    return [], relaxed

```

Worked example — the model asks for a beginner kettlebell hip-extension glute exercise and the catalog has no kettlebell entries:

```

relaxed = ['movement_pattern', 'max_difficulty', 'equipment']
rows = 2    (first: Bodyweight Glute Bridge)

```

The tool result then begins:

NOTE: no exact match. Relaxed filters: movement_pattern, max_difficulty, equipment.
Tell the user what you widened.

3.7 Serializing candidates for the prompt

Pipe-delimited lines, not JSON — same information, roughly 40% fewer tokens, and models follow the format reliably. On a GPU-time meter those saved tokens are saved money on every single turn.

```
def card_line(r) -> str:
    parts = [f"id={r['id']}", r["name"]]
    if r["name_ar"]:
        parts.append(r["name_ar"])
    parts += [f"{r['muscle_group']}/{r['prime_mover']}", r["difficulty"],
              r["equipment"], r["body_region"], r["mechanics"],
              r["movement_patterns"] or "-"]
    return " | ".join(str(p) for p in parts)
```

Real output from the databases:

```
CANDIDATE EXERCISES (cite by id, never invent an id):
id=bodyweight-glute-bridge | Bodyweight Glute Bridge | جسر الأرداف بوزن الجسم |
  Glutes/Gluteus Maximus | Beginner | Bodyweight | Lower Body | Compound | Hip Extension
id=bodyweight-hip-thrust | Bodyweight Hip Thrust | Glutes/Gluteus Maximus | Novice |
  Bodyweight | Lower Body | Compound | Hip Extension
```

Deliberately excluded from the card: demo_url, explain_url, grip, foot_elevation, load_position_ending, plane_of_motion_2/3, posture in most cases. URLs in particular are never shown to the model — it cannot then produce a broken one, and the app attaches the real link from its local database at render time.

For meals, include the estimate flag so the model's language stays honest:

```
CANDIDATE MEALS (macros are per-serving ESTIMATES):
id=52772 | Chicken Handi | Chicken/Indian | ~94g protein | est=yes conf=0.80 | Curry
id=52774 | Grilled Salmon Salad | Seafood/Japanese | ~24g protein | est=yes conf=1.00
```

3.8 Turn B — schema-constrained output

Ollama's format parameter accepts a JSON Schema and constrains decoding to it. This is categorically more reliable than asking for JSON in the prompt, and it behaves identically on cloud models.

```
{
  "type": "object",
  "required": ["reply"],
  "additionalProperties": false,
  "properties": {
    "reply": {"type": "string",
              "description": "Plain prose in the user's language. No markdown, no URLs, no ids."},
    "exercise_refs": {"type": "array", "maxItems": 6, "items": {"type": "string"}},
    "meal_refs": {"type": "array", "maxItems": 6, "items": {"type": "string"}},
    "action": {
      "type": "object",
      "required": ["type"],
      "properties": {
```

```

        "type": {"type": "string",
                  "enum": ["open_exercise", "open_meal", "open_filtered_list",
                           "start_workout", "none"]},
        "payload": {"type": "object"}
    },
    "safety_flag": {"type": "string",
                    "enum": ["none", "medical", "injury", "nutrition_extreme", "off_topic"]}
}

```

`reply` carries prose only. Every navigable thing lives in the ref arrays and `action`. Because nothing is embedded in the prose, a malformed reference can never corrupt the text the user reads.

Verify this on your first day of integration. Structured output and tool calling are the two capabilities this entire design rests on. Both are documented to work identically for cloud models, but "documented" and "verified against `gemma4:31b` on your account" are different things. Phase 0a of §7.8 is a 20-line script that calls `/api/chat` twice — once with `tools`, once with `format` — and asserts the shape of what comes back. Write it before you write the gateway.

3.9 Validating what comes back

```

def validate_refs(model_refs, candidate_ids, con):
    """refs n this turn's candidates n the catalog. Both intersections matter."""
    seen, kept, dropped = set(), [], []
    for ref in model_refs or []:
        if ref in seen:
            continue
        seen.add(ref)
        if ref not in candidate_ids:
            dropped.append((ref, "not_in_candidate_set"))
            continue
        row = con.execute(
            "SELECT id, name, name_ar, muscle_group, difficulty, demo_url"
            " FROM exercise_card WHERE id = ?", (ref,)).fetchone()
        if row is None:
            dropped.append((ref, "not_in_catalog"))
            continue
        kept.append(dict(row))
    return kept, dropped

```

- `n candidate_ids` kills hallucinated-but-plausible slugs — the model may only cite what retrieval actually handed it this turn.
- `n catalog` catches ids that exist in the candidate set but have since been removed.

Log every drop with the model's raw output. A rising drop rate is your earliest signal that a prompt or retrieval change regressed — or, now, that the hosted model behind your tag changed underneath you (§7.7).

3.10 Worked example

User (English): *"beginner glute exercises, I only have a mat"*

Turn A tool call:

```
{"name": "search_exercises",
  "arguments": {"muscle_group": "Glutes", "equipment": "Bodyweight",
               "max_difficulty": 2, "limit": 6}}
```

SQL returns 2 rows; candidate set is `{bodyweight-glute-bridge, bodyweight-hip-thrust}`.

Turn B output:

```
{
  "reply": "With just a mat, glute bridges are the place to start – lie on your back, feet flat, and drive through your heels until your hips are level with your knees. Three sets of 12, squeezing for a second at the top. Once that feels easy, single-leg hip thrusts add load without any equipment.",
  "exercise_refs": ["bodyweight-glute-bridge", "bodyweight-hip-thrust"],
  "action": {"type": "none"},
  "safety_flag": "none"
}
```

Both refs validate. The app renders two cards below the text.

User (Arabic): "تمارين بطن للمبتدئين"

```
{"name": "search_exercises",
  "arguments": {"muscle_group": "Abdominals", "max_difficulty": 2, "limit": 6}}
```

The model replies in Arabic, names each exercise in Arabic with the English in parentheses, and returns the same slug ids — ids are language-independent, which is precisely why deep linking works identically in both locales.

3.11 Context budget

There is no `num_ctx` to set on a cloud model, and `gemma4:31b` serves 256K, so context is not a ceiling you can hit. Budget anyway — this is a **cost** discipline rather than a capacity one:

Segment	Tokens
System prompt + vocabulary	~600
Tool schemas (turn A only)	~250
Rolling history — last 6 turns	~1,200
Rolling summary of older turns	~200
Candidate cards (≤ 8)	~700
User message	~100
Reserve for the reply	~600
Total	~3,650

Turn A sends the system prompt, vocabulary, tool schemas and history; turn B sends the same minus the tool schemas, plus the candidate cards. Neither turn sends all ~3,650 at once, but between them you pay for most of it twice — every token in that table is prefill work on somebody's GPU, billed to you. A 256K

window is an invitation to stop compressing history; resist it. Doubling the prompt roughly doubles prefill time, which is also the largest controllable component of time-to-first-token.

Memory strategy. Keep the last 6 turns verbatim. When history exceeds ~1,500 tokens, compress everything older into a ≤150-token profile summary — goals, injuries, preferences, current program — and persist it with the conversation. Send the summary in place of the dropped turns.

3.12 Ollama Cloud request configuration

```
POST https://ollama.com/api/chat
Authorization: Bearer $OLLAMA_API_KEY
Content-Type: application/json

{
  "model": "gemma4:31b",
  "messages": [ ... ],
  "stream": true,
  "think": false,
  "options": {
    "temperature": 0.6,
    "top_p": 0.9,
    "repeat_penalty": 1.05,
    "num_predict": 512
  }
}
```

Four fields carry the weight, and two that you may expect are deliberately absent:

Field	Value	Why
<code>model</code>	<code>gemma4:31b</code>	See §3.13. Use the <code>-cloud</code> suffix only when proxying through a local daemon
<code>think</code>	<code>false</code>	Required. See below
<code>stream</code>	<code>true</code> on turn A <code>B</code>	Turn A returns a tool call and nothing worth streaming
<code>options.num_predict</code>	<code>512</code>	A direct spending cap, not just a length cap
<code>keep_alive</code>	omit	Not supported on cloud models. Residency is the provider's problem
<code>options.num_ctx</code>	omit	Not supported on cloud models; the service sets the window

Sending the two unsupported options is not an error — they are simply ignored. Leave them out anyway, so a reader of the request log does not believe a setting is in force when it isn't.

On `think`. `gemma4` supports configurable reasoning — `true / false`, or `"low" | "medium" | "high" | "max"`. When enabled, the reasoning trace arrives in a separate `message.thinking` field rather than `message.content`, so it will not corrupt the `ReplyStreamer` in §4.4. What it will do is add seconds to time-to-first-token and GPU-seconds to your bill, producing a trace this system throws away. Send `"think": false` explicitly on both turns; do not rely on the default.

The one case worth measuring is turn A. If the eval set (§7.5) shows the model picking wrong filters on constrained questions — "glute work, no equipment, bad lower back" — try `"think": "low"` on turn A

only, and compare tool-argument accuracy against the extra latency. Turn B should stay `false` regardless: it is a writing task against a fixed candidate set, and it is the turn the user is watching.

3.13 Model choice — gemma4:31b

Serving the model yourself makes size a hardware question: whatever fits in the VRAM you have. Serving it on cloud makes size a metering question instead — a larger model does not need a larger machine, it just runs the meter faster. That frees the choice to be made on quality, which for a bilingual app means Arabic.

Property	<code>gemma4:31b</code>	What it means here
Parameters	31B dense	Comfortably larger than anything you would run on a single mid-range GPU. Arabic is where you should expect to feel it (§7.7)
Context	256K	Far more than this design needs. Do not treat it as a licence to stop compressing (§3.11)
Tools	Yes	Required by turns A. Verify on day one
Structured output	Yes, via <code>format</code>	Required by turn B
Thinking	Configurable	Must be explicitly disabled (§3.12)
Vision	Yes	Unused. Do not send images — you would pay for tokens the design has no use for. A future "photograph your meal" feature is the only reason to revisit this
Sizes available	12b, 26b, 31b	If the meter runs hotter than you like, <code>gemma4:12b</code> for turn A and <code>31b</code> for turn B is the obvious split — turn A only has to pick filters

Two things to verify before you commit.

- **Tool-call parsing.** There was a known parsing failure on `gemma4` tool calls (`invalid character 'p' after object key:value pair`) that was fixed in Ollama shortly after 0.20.0-rc1. If you route through a local Ollama daemon as a proxy rather than calling `ollama.com` directly, pin a version past that fix. Calling the cloud endpoint directly means the provider owns that parser version rather than you — which removes it from your deploy checklist, but does not make it impossible. This is one of the things the deploy-time assertion in §7.3 exists to catch.
- **Tool-result role naming.** At least one third-party proxy has hit an infinite tool loop with `gemma4` caused by a mismatch between the `tool` role and a `tool_responses` role. If you put any middleware — LiteLLM, an OpenAI-compat shim, a gateway framework — between yourself and `ollama.com`, assert that a tool result round-trips correctly before building on it. The reference loop in §4.5 speaks native Ollama and appends `{"role": "tool", ...}` directly, which is the shape the API documents.

Benchmark before committing. Run 30 of your own real questions, half English and half Arabic, against `gemma4:31b` and against at least one alternative. This is cheap here in a way it is not on your own hardware — swapping models is a string in a config file, not a re-provisioning — so there is no excuse for choosing on reputation.

3.14 The cost model: GPU-time, not tokens

Ollama Cloud meters GPU-time, not tokens or requests. Three things move the meter: the weight class of the model, how long it generates, and how much prompt it has to read. That maps onto four levers you already have:

Lever	Where	Effect
The fast path	\$4.7	Removes turn A entirely on unambiguous messages: one fewer round trip, one fewer prefill of the prompt and tool schemas, one fewer occupancy of the concurrency slot. Against the \$1.5 budget that is roughly 20–35% of GPU-time per message, and a much larger share of throughput. The highest-leverage optimisation in the system
<code>num_predict:</code> 512	\$3.12	Hard ceiling on the expensive half of every request
History compression	\$3.11	Bounds prefill on long conversations
<code>think: false</code>	\$3.12	Removes an entire hidden generation pass

You cannot read your remaining quota from the API. There is no usage endpoint and no usage header — this is a known and requested gap. Two consequences:

1. **Estimate spend yourself.** Log `tokens_in`, `tokens_out` and wall-clock upstream duration per turn (§4.9). Upstream duration is the closest proxy you have for GPU-seconds. Track it per user and in aggregate; a per-1,000-messages figure is what you will need when someone asks what the chatbot costs.
2. **Detect exhaustion reactively.** A 429 from `ollama.com` is your only reliable signal that a window is spent. Handle it as a first-class state (§4.10), not as a generic upstream error — the correct user-facing behaviour for "we are out of quota until the window resets" is degraded mode (§4.11), not a retry chip that will fail again immediately.

Free-plan arithmetic. One concurrent cloud model means one in-flight generation. A two-turn message occupies that slot for both turns. At ~2–4 seconds of occupancy per message, a single free slot serves on the order of 15–30 messages per minute *if nothing else queues* — and that is throughput, not quota, which will bind first. This is fine for development and for an internal beta. It is not a launch configuration.

4. The gateway

4.1 Responsibilities

Concern	Why it lives here
System prompt + vocabulary	Iterate without an app release
Tool definitions	Must stay in sync with the SQL
Retrieval SQL	Deterministic, auditable, uses the server's DB copy
Ref validation	Client-side validation is not validation
The Ollama API key	A key in the app binary is a key in the wild
Upstream concurrency	Your plan allows N in-flight generations. Something has to enforce N
Quota accounting	The API won't tell you what you've spent. Only you can measure it
Auth, rate limit, per-user quota	Ollama Cloud authenticates your account, not your users
Streaming JSON → SSE	Keeps partial-JSON parsing out of Dart
Logging	Your only feedback loop for improving retrieval — and now, for costing it

4.2 Endpoints

POST /v1/chat	SSE stream. The only endpoint the chat screen calls.
GET /v1/catalog/manifest	{data_version, sha256 per db} – version handshake
GET /healthz	liveness + upstream reachability + key validity

Request body — unchanged:

```
{
  "conversation_id": "c_01H...",
  "message": "beginner glute exercises, I only have a mat",
  "locale": "en",
  "profile": {
    "level": "Beginner", "goal": "general fitness",
    "equipment": ["Bodyweight"], "limits": ["lower back pain"],
    "age_band": "25-34", "units": "metric"
  },
  "data_version": "2026.07.25"
}
```

/healthz answers three separate questions. Can I reach `ollama.com`, is my key still valid, and is the model still listed — three failures with three different pages to wake someone for, and they must be distinguishable in the response. Use `GET /api/tags`, which is cheap and consumes no GPU-time:

```
_upstream_cache = {"at": 0.0, "checks": None}    # see the caching note below

async def healthz() -> JSONResponse:
```

```

checks = {"db": False, "upstream": False, "key": False, "model": False}
try:
    checks["db"] = ex_con.execute("SELECT 1 FROM meta LIMIT 1").fetchone() is not None
except sqlite3.Error:
    pass

now = time.monotonic()
if _upstream_cache["checks"] and now - _upstream_cache["at"] < 60:
    checks.update(_upstream_cache["checks"])    # don't probe on every scrape
else:
    up = {"upstream": False, "key": False, "model": False}
    try:
        async with httpx.AsyncClient(timeout=5) as c:
            r = await c.get(f"{OLLAMA}/api/tags", headers=HEADERS)
            up["upstream"] = True
            up["key"] = r.status_code not in (401, 403)
            if r.status_code == 200:
                names = {m.get("name", "") for m in r.json().get("models", [])}
                up["model"] = any(x.startswith(MODEL) for x in names)
    except (httpx.RequestError, ValueError):
        pass
    _upstream_cache.update(at=now, checks=up)
    checks.update(up)

ok = all(checks.values())
return JSONResponse(
    {"ok": ok, "checks": checks, "degraded": checks["db"] and not ok},
    status_code=200 if ok else 503,
)

```

Note `degraded`: the database is local and the model is not. A gateway that can still retrieve is not fully down — it is in the state §4.11 describes, and the health check should say so rather than reporting a flat failure.

Three details that are easy to get wrong. The database check is inside its own `try`, because a health endpoint that returns 500 tells your load balancer nothing. The upstream result is cached for 60 seconds, because a liveness probe scraped every 10 seconds otherwise generates thousands of authenticated calls a day against an account whose usage you cannot read. And `GET /api/tags` is the right probe precisely because it consumes no GPU-time — never health-check with a chat completion.

4.3 The SSE protocol

Four event types. The app needs no JSON parsing beyond `json.decode` per event.

```

event: token
data: {"t":"With just a mat, glute bridges are "}

event: token
data: {"t":"the place to start – lie on your back..."}

event: refs
data: {"exercise_refs":["bodyweight-glute-bridge","bodyweight-hip-thrust"],
      "meal_refs":[],
      "action":{"type":"none"},
      "snapshots":[{"id":"bodyweight-glute-bridge",
                    "name":"Bodyweight Glute Bridge",
                    "name_ar":"جسر الأرداف بوزن الجسم",
                    "muscle_group":"Glutes","difficulty":"Beginner"}]}

event: done
data: {"reply_chars":1043,"latency_ms":2870,"data_version":"2026.07.25",

```



```
"dropped_refs":0,"degraded":false}

event: error
data: {"code":"quota","message":"The coach is resting. Here's what I found.",
      "retryable":false}
```

`snapshots` is the denormalized fallback: if the app's database is older than the gateway's and doesn't contain an id, it still renders a card from this payload — just a non-tappable one.

The `degraded` flag in `done` tells the app that the cards below the message came from retrieval alone, with no coach commentary (§4.11). Note that `reply_chars` is a character count, not a token count — do not divide a bill by it.

4.4 Streaming the `reply` field without parsing partial JSON

Turn B emits a JSON object token by token. Waiting for it to close before showing anything costs the user 2–3 seconds of blank screen. Parsing half-written JSON is a classic source of bugs.

The trick: you don't need a JSON parser. You need to find the `"reply"` string value and stream its contents, handling escapes, until it closes.

```
class ReplyStreamer:
    """Incrementally extract the "reply" string from a JSON object as it arrives.

    Feed it raw chunks; it yields decoded prose. Everything after the reply string
    closes is buffered and parsed normally once the object completes.
    """

    def __init__(self):
        self.buf = ""           # everything seen so far
        self.pos = 0           # scan position inside the reply string
        self.started = False    # have we found the opening quote of the value?
        self.done = False      # has the reply string closed?

    def feed(self, chunk: str) -> str:
        self.buf += chunk
        if self.done:
            return ""
        if not self.started:
            m = re.search(r'"reply"\s*:\s*', self.buf)
            if not m:
                return ""
            self.started = True
            self.pos = m.end()
        out, i = [], self.pos
        while i < len(self.buf):
            ch = self.buf[i]
            if ch == "\\":
                if i + 1 >= len(self.buf):
                    break                # escape split across chunks — wait
                nxt = self.buf[i + 1]
                if nxt == "u":
                    if i + 6 > len(self.buf):
                        break            # \uXXXX split across chunks — wait
                    out.append(json.loads(f'"{self.buf[i:i + 6]}"'))
                    i += 6
                else:
                    out.append({"n": "\n", "t": "\t", "r": "\r", "b": "\b",
                               "f": "\f"}.get(nxt, nxt))
                    i += 2
            continue
```

```

        if ch == '\u':
            self.done = True
            i += 1
            break
        out.append(ch)
        i += 1
    self.pos = i
    return "".join(out)

def final(self) -> dict:
    """Parse the complete object once the stream ends."""
    return json.loads(self.buf)

```

The escape handling matters: Arabic arrives as literal UTF-8 from Ollama, but a model may still emit `\u`, and a `\uXXXX` sequence split across two chunks would otherwise corrupt the output. The `break`s leave the partial escape in the buffer for the next `feed()`.

One cloud-specific note: the streamer reads `message.content` only. With `think` enabled, reasoning arrives in `message.thinking` and would be silently discarded — the output stays correct, but the user watches a blank bubble for however long the model reasons. That is the second reason §3.12 insists on `think: false`.

4.5 The two-turn loop

```

import httpx, json, os, time

OLLAMA = "https://ollama.com"
MODEL = "gemma4:31b"
HEADERS = {"Authorization": f"Bearer {os.environ['OLLAMA_API_KEY']}" }
OPTIONS = {"temperature": 0.6, "top_p": 0.9,
           "repeat_penalty": 1.05, "num_predict": 512}

async def chat(req, ex_con, ml_con, emit):
    """emit(event: str, data: dict) writes one SSE frame."""
    t0 = time.monotonic()
    messages = build_messages(req)  # system + vocabulary + history + user

    # ---- turn A: let the model decide what to look up -----
    # post_json takes the concurrency slot itself, per attempt (§4.6), and has
    # released it by the time this returns – retrieval never holds a slot open.
    turn_a = (await post_json("/api/chat", {
        "model": MODEL, "messages": messages, "tools": TOOLS,
        "stream": False, "think": False, "options": OPTIONS,
    }))[ "message" ]

    candidate_ids, tool_blocks = set(), []
    for call in turn_a.get("tool_calls", []):
        fn = call["function"]["name"]
        args = call["function"]["arguments"]
        if fn == "search_exercises":
            rows, relaxed = search_exercises(ex_con, **sanitize(args))
            candidate_ids |= {r["id"] for r in rows}
            tool_blocks.append(render_exercise_candidates(rows, relaxed))
        elif fn == "search_meals":
            rows, relaxed = search_meals(ml_con, **sanitize(args))
            candidate_ids |= {r["id"] for r in rows}
            tool_blocks.append(render_meal_candidates(rows, relaxed))
        elif fn == "search_by_text":

```

```

        ex_rows, ml_rows = fts_search(ex_con, ml_con, **sanitize(args))
        candidate_ids |= {r["id"] for r in ex_rows} | {r["id"] for r in ml_rows}
        if ex_rows:
            tool_blocks.append(render_exercise_candidates(ex_rows, []))
        if ml_rows:
            tool_blocks.append(render_meal_candidates(ml_rows, []))

    if turn_a.get("tool_calls"):
        messages.append(turn_a)
        messages.append({"role": "tool", "content": "\n\n".join(tool_blocks)})

# ---- turn B: write the answer, constrained to the output schema -----
streamer, chars = ReplyStreamer(), 0
async with upstream_slot():
    async with httpx.AsyncClient(timeout=120) as client:
        async with client.stream("POST", f"{OLLAMA}/api/chat",
                                headers=HEADERS, json={
                                    "model": MODEL, "messages": messages, "stream": True,
                                    "format": OUTPUT_SCHEMA, "think": False, "options": OPTIONS,
                                }) as resp:
            raise_for_upstream(resp) # $4.6 - same mapping as post_json
            async for line in resp.aiter_lines():
                if not line.strip():
                    continue
                piece = json.loads(line)["message"]["content"]
                text = streamer.feed(piece)
                if text:
                    chars += len(text)
                    await emit("token", {"t": text})

# ---- validate and finish -----
try:
    final = streamer.final()
except json.JSONDecodeError:
    final = {"reply": "", "exercise_refs": [], "meal_refs": []}

kept_ex, dropped = validate_refs(final.get("exercise_refs"), candidate_ids, ex_con)
kept_ml, d2 = validate_meal_refs(final.get("meal_refs"), candidate_ids, ml_con)

await emit("refs", {
    "exercise_refs": [r["id"] for r in kept_ex],
    "meal_refs": [r["id"] for r in kept_ml],
    "action": final.get("action", {"type": "none"}),
    "snapshots": kept_ex + kept_ml,
})
await emit("done", {
    "reply_chars": chars, "latency_ms": int((time.monotonic() - t0) * 1000),
    "data_version": DATA_VERSION, "dropped_refs": len(dropped) + len(d2),
    "degraded": False,
})
log_turn(req, turn_a, candidate_ids, final, dropped + d2)

async def handle_chat(req, ex_con, ml_con, emit):
    """The only entry point. Every upstream failure lands in degraded mode."""
    try:
        await chat(req, ex_con, ml_con, emit)
    except Busy:
        await emit("error", {"code": "busy",
                             "message": COPY[loc(req)]["busy"], "retryable": True})
    except QuotaExhausted:
        breaker.trip()
        await chat_degraded(req, ex_con, ml_con, emit, "quota")
    except BadCredentials:
        alert("ollama credentials rejected")
        await chat_degraded(req, ex_con, ml_con, emit, "upstream")

```

```
except (UpstreamDown, httpx.HTTPError):
    await chat_degraded(req, ex_con, ml_con, emit, "upstream")
```

Six details in that loop are load-bearing:

- Every call carries `HEADERS`. There is no unauthenticated path to `ollama.com`, and no default that will save you if you forget.
- `"think": False` is explicit on both calls rather than left to the default (§3.12).
- Both calls pass through the concurrency gate from §4.6, but at different levels. Turn A goes through `post_json`, which takes and releases the slot itself on each attempt — so a backoff never idles the account's only slot, and the SQL retrieval that follows never holds one open. Turn B streams, so it holds the slot explicitly for the duration of the generation. **Do not wrap a `post_json` call in `upstream_slot()` as well** — with `MAX_INFLIGHT=1` that is a self-deadlock, and it is the easiest mistake to make in this file.
- `post_json` replaces the bare `client.post` so retries and error mapping (§4.6) live in one place.
- `raise_for_upstream(resp)` on the streaming call rather than a bare `raise_for_status()`. A 429 arrives as an HTTP status *before* any body; mapping it to the same typed exceptions `post_json` raises is what makes the quota path in §4.10 reachable from turn B — which is the longer and more likely-throttled of the two turns. Silently iterating an error response produces an empty reply instead of a handled failure.
- `handle_chat` wraps the whole thing. **Nothing calls `chat` directly**. Every typed upstream failure lands in `chat_degraded` (§4.11) rather than in a stack trace.
- `chars` counts characters, not tokens. Name it that way in the log and in the SSE payload, or somebody will eventually cost the feature with it.

`sanitize(args)` drops keys the tool schema doesn't define and coerces types — a model will occasionally return `"max_difficulty": "2"` as a string.

4.6 The upstream client — key, concurrency, retries

Everything that talks to `ollama.com` goes through one module. Keeping it that way is what stops credential handling, concurrency and retry policy from leaking into the chat logic — and it is the file to hand a reviewer first.

```
import asyncio, httpx, os, random
from contextlib import asynccontextmanager

OLLAMA      = os.environ["OLLAMA_HOST"]          # https://ollama.com
MODEL       = os.environ["OLLAMA_MODEL"]        # gemma4:31b
HEADERS     = {"Authorization": f"Bearer {os.environ['OLLAMA_API_KEY']}"}
MAX_INFLIGHT = int(os.environ.get("OLLAMA_MAX_INFLIGHT", "1")) # your plan's allowance
QUEUE_DEPTH = int(os.environ.get("OLLAMA_QUEUE_DEPTH", "8"))

_slots      = asyncio.Semaphore(MAX_INFLIGHT)
_inflight   = 0                                     # admitted: waiting for a slot, or holding one

class Busy(Exception):
    pass # queue full — tell the app to retry
class QuotaExhausted(Exception):
    pass # 429 from upstream
class UpstreamDown(Exception):
    pass # 5xx, timeout, connection error
class BadCredentials(Exception):
    pass # 401/403 — page someone
class UpstreamRejected(Exception):
    pass # 4xx we caused; a bug, not an incident
```

```

def raise_for_upstream(r: httpx.Response) -> None:
    """One mapping from HTTP status to typed error. Used by both turns."""
    if r.status_code in (401, 403):
        raise BadCredentials(f"{r.status_code}")    # never retry – it won't help
    if r.status_code == 429:
        raise QuotaExhausted(r.headers.get("retry-after", ""))
    if r.status_code >= 500:
        raise UpstreamDown(f"{r.status_code}")
    if r.status_code >= 400:
        raise UpstreamRejected(f"{r.status_code}")    # malformed tools/format – fix it

@asynccontextmanager
async def upstream_slot():
    """Bounded queue in front of the plan's concurrency allowance."""
    global _inflight
    if _inflight >= QUEUE_DEPTH:
        raise Busy()    # no await between the check and the
                        # increment, so this needs no lock
    _inflight += 1
    try:
        async with _slots:
            yield
    finally:
        _inflight -= 1

async def post_json(path: str, body: dict, attempts: int = 2) -> dict:
    """Call OUTSIDE upstream_slot(): it acquires the slot per attempt, so a
    backoff never idles the account's only concurrency slot."""
    last = None
    for n in range(attempts):
        try:
            async with upstream_slot():
                r = await _client.post(f"{OLLAMA}/{path}", headers=HEADERS, json=body)
                raise_for_upstream(r)
                return r.json()
        except (UpstreamDown, httpx.RequestError) as e:
            last = e
            if n + 1 < attempts:
                await asyncio.sleep((0.4 * 2 ** n) + random.uniform(0, 0.2))
    raise UpstreamDown(str(last))

_client = httpx.AsyncClient(timeout=30)    # one client, reused; not one per call

```

Sizing `MAX_INFLIGHT`. Set it to your plan's concurrent-model allowance — 1 on the Free plan. Making it an environment variable is the whole point: upgrading the plan should be a deploy, not a patch. Setting it higher than your plan allows does not buy throughput; it converts your own orderly queue into upstream 429s.

This semaphore is per process, and your plan's allowance is per account. Two workers, or two replicas behind a load balancer, give you $2 \times \text{MAX_INFLIGHT}$ in flight upstream — exactly the situation the gate exists to prevent. Until you are on a plan whose allowance comfortably exceeds your replica count, either run a single process, or set `MAX_INFLIGHT = plan_allowance / replica_count` and accept the rounding, or move the gate into Redis. Whichever you pick, **write it down next to the deployment config**, because the day someone scales the gateway to two pods is the day the 429 rate goes up for no visible reason.

Why a bounded queue and not an unbounded one. With one slot and a 3-second message, a queue of 30 means the 30th user waits 90 seconds for an answer they stopped caring about 80 seconds ago.

`QUEUE_DEPTH` counts everything admitted — the request holding the slot plus those waiting — so 8 means at most 7 waiters and a worst-case wait around 21 seconds on a single slot. That is already too long; treat 8 as a ceiling you tune down, not up. Past it, `Busy` produces an honest "the coach is busy, try again" chip within a second, which is a better experience than a spinner that eventually times out.

Retries, carefully. Retry connection errors and 5xx; never retry 401/403 (the key is wrong and will still be wrong) and never blindly retry 429 (you are out of allowance and retrying spends what's left of it on failures). Never retry a 4xx you caused — `UpstreamRejected` means your `tools` or `format` payload is malformed, and it should page you, not loop.

Turn B is a separate case. By the time it fails you may already have streamed tokens into the user's bubble, and replaying them would duplicate text. **Retry turn B only when `chars == 0`** — that single condition is what makes the "retry once at `temperature 0.3`" row in §4.10 safe, and it is the rule to state in the code review.

4.7 The fast path

Two Ollama round trips is the safe default, but many messages are unambiguous. When a cheap classifier on the gateway (or in Dart, against the local DB) confidently extracts facets, run retrieval *before* turn A, inject the candidates directly and skip straight to turn B.

The classifier's output must be **vocabulary values and tool-argument names**, not the words the user typed. Retrieval filters on exact strings (`'Glutes'` , `'Bodyweight'`) and on `max_difficulty` as an integer 1–8; a classifier that returns `"glute"` and `"difficulty"` silently matches nothing, the relaxation ladder strips every filter, and the user gets the globally most popular exercises. Map, don't pass through:

```
# Right-hand sides are exactly what search_exercises() accepts. Build the first
# two dicts from the database at boot, next to the vocabulary block (§3.2), so
# they cannot drift from what retrieval will match.
MUSCLE_WORDS = {
    "chest": "Chest", "glute": "Glutes", "glutes": "Glutes",
    "ab": "Abdominals", "abs": "Abdominals", "core": "Abdominals",
    "back": "Back", "shoulder": "Shoulders", "shoulders": "Shoulders",
    "bicep": "Biceps", "biceps": "Biceps", "tricep": "Triceps", "triceps": "Triceps",
    "leg": "Quadriceps", "legs": "Quadriceps", "quad": "Quadriceps",
    "hamstring": "Hamstrings", "calf": "Calves", "calves": "Calves",
}
EQUIP_WORDS = {
    "bodyweight": "Bodyweight", "no equipment": "Bodyweight",
    "at home": "Bodyweight", "mat": "Bodyweight",
    "dumbbell": "Dumbbell", "barbell": "Barbell", "cable": "Cable",
}
DIFF_WORDS = {"beginner": 1, "novice": 2, "intermediate": 3, "advanced": 4}

QUESTION_RE = re.compile(
    r"\b(why|how|should i|is it|am i|can i|hurt|pain|safe|program|plan)\b", re.I)

def classify(message: str) -> dict | None:
    """Return tool arguments, or None to mean 'use the full two-turn loop'."""
    m = message.lower()
    if QUESTION_RE.search(m) or len(m) > 120:
        return None # technique / safety / programming
    facets = {}
    for word, value in MUSCLE_WORDS.items():
        if re.search(rf"\b{word}\w*\b", m):
            facets["muscle_group"] = value
            break
```

```

for word, value in EQUIP_WORDS.items():
    if word in m:
        facets["equipment"] = value
        break
for word, rank in DIFF_WORDS.items():
    if word in m:
        facets["max_difficulty"] = rank
        break
return facets if "muscle_group" in facets else None

```

If `classify` returns facets, skip turn A: run retrieval directly, inject the candidates, go straight to turn B. If it returns `None` — no muscle group, or the message reads like a question about technique, safety or programming — use the full loop. Log which path each turn took so you can measure the hit rate.

Keep the word maps generated from the database alongside the vocabulary block. The moment they are hand-maintained they drift, and the failure is invisible: no error, just a fast path that quietly stops matching and a cost line that quietly goes up.

Treat this as required, not as an optimisation to add later. Skipping turn A removes one full upstream round trip, one prefill of the system prompt and tool schemas, and one occupancy of your single concurrency slot. Against the \$1.5 budget that is roughly a fifth to a third of GPU-time per message, and a far larger share of throughput — on one slot, halving the requests roughly doubles the users you can serve. It is simultaneously the largest latency win, the largest cost win and the largest throughput win available in this design. Build it in phase 3, not phase 6.

4.8 Auth, rate limiting and quota

There are now two distinct budgets to protect: your users' fair access to the service, and your account's finite allowance upstream. They need separate mechanisms.

Protecting the key

- The key lives in a secret manager and reaches the process as an environment variable. Never in the repo, never in an image layer, never in a log line, never in an error message returned to the app.
- Keys do not expire on their own. Rotation is manual: create the new key, deploy it, then revoke the old one at ollama.com/settings/keys. Do it on a schedule and on every person-leaves-the-team event.
- Assert at boot that `OLLAMA_API_KEY` is present and non-empty. A gateway that starts without a key and fails on the first user message is worse than one that refuses to start.

Protecting your users from each other

- Verify the app's JWT (Firebase, Supabase, whatever you already use) on every request. Reject anonymous traffic — an open proxy to a paid inference account is a very expensive thing to leave running.
- Two limits: a short-window rate limit and a rolling monthly quota per user. Size the short window against your *account* throughput, not against what feels generous — \$3.14 puts a single free concurrency slot at roughly 15–30 messages/minute for *everyone combined*, so a 20/minute per-user limit lets one person consume the entire service. Something around 6/minute is defensible while `MAX_INFLIGHT` is 1; raise it with the plan. Return `429` with a `Retry-After`; the app shows a friendly chip.
- Cap `message` length (~2,000 chars) and history length server-side. Both are prefill you pay for.

Protecting your account from your users

- The `upstream_slot()` gate in §4.6 is the hard limit. Nothing reaches `ollama.com` without passing through it.
- Track estimated spend per user from logged upstream duration (§3.14). The first sign of abuse is usually one `user_id` with an order-of-magnitude outlier.
- Set a global daily circuit breaker — an internal number, not one the API gives you. When aggregate upstream seconds for the day cross your threshold, drop to degraded mode (§4.11) for everyone rather than discovering at 2 a.m. that the window is spent.

4.9 Logging

Log one row per turn. This is the only way you will ever improve retrieval — and now the only way you will ever cost the feature.

```
{
  "ts": "2026-07-25T14:03:11Z",
  "user": "u_88f",
  "conversation": "c_01H",
  "locale": "en",
  "path": "two_turn",
  "prompt_hash": "9f2c...",
  "model": "gemma4:31b",
  "tools": [
    {
      "name": "search_exercises",
      "args": {
        "muscle_group": "Glutes",
        "equipment": "Bodyweight",
        "max_difficulty": 2
      },
      "rows": 2,
      "relaxed": []
    }
  ],
  "candidate_ids": ["bodyweight-glute-bridge", "bodyweight-hip-thrust"],
  "refs_returned": ["bodyweight-glute-bridge", "bodyweight-hip-thrust"],
  "refs_dropped": [],
  "safety_flag": "none",
  "tokens_in": 3610,
  "tokens_out": 214,
  "reply_chars": 1043,
  "latency_ms": 2870,
  "ttft_ms": 1240,
  "upstream_ms_a": 480,
  "upstream_ms_b": 1210,
  "queue_ms": 0,
  "upstream_status": 200,
  "attempts": 1,
  "degraded": false
}
```

`upstream_ms_a` and `upstream_ms_b` are your GPU-time proxy; `queue_ms` tells you whether your concurrency allowance is the bottleneck; `upstream_status` and `attempts` tell you how often the provider is failing you.

Metric	Healthy	What a regression means
<code>refs_dropped</code> rate	< 2%	Prompt or retrieval regression — or the hosted model changed under your tag
Empty-candidate rate	< 5%	Vocabulary drift, or the catalog genuinely lacks coverage
<code>relaxed</code> non-empty rate	< 15%	Users asking for equipment you don't have data for
p95 TTFT	< 2.5 s	Upstream slowness, or queueing. Set the real threshold from your own measured p50 once §1.5 has been replaced with data
p95 <code>queue_ms</code>	near 0	Your plan is the bottleneck. With one slot there is no middle ground — either nothing queues, or a waiter pays a full generation. Any sustained non-zero p95 means raise <code>MAX_INFLIGHT</code> and the plan behind it
Upstream 429 rate	~0	Quota window spent, or you are exceeding plan concurrency
Upstream 5xx rate	< 0.5%	Provider incident. Confirm degraded mode is engaging
Upstream seconds / 1k messages	trend flat	Rising means prompt bloat or history compression stopped working

4.10 Error handling

Failure	Gateway behaviour	What the user sees
Model emits non-JSON despite <code>format</code>	Retry turn B once at <code>temperature 0.3</code> only if <code>chars == 0</code> ; otherwise send what streamed, with zero refs	A normal answer, no cards
Ref not in candidate set	Drop, log, keep prose	Fewer cards; text unchanged
Retrieval returns 0 rows	Relax filters, note it in the tool result	"I widened the search — here's what I have"
401 / 403 from ollama.com	Do not retry. Page on-call. Serve degraded mode	Cards with no commentary — the app looks like search, not like an error
429 from ollama.com	Do not retry. Trip the breaker for the window. Serve degraded mode	Cards, plus "the coach is resting" — no retry chip, because retrying will fail
Upstream 5xx / timeout	One backoff retry on turn A; on turn B only if no tokens sent. Then degraded mode	Cards; retry chip; the typed message is preserved
Local queue full (Busy)	<code>event: error</code> , <code>retryable: true</code> , immediately	Retry chip within a second, not after a timeout
Tool args fail validation	Coerce what's coercible, drop the rest, run with what remains	Nothing — degrades silently
App <code>data_version</code> older	Attach <code>snapshots</code> ; set a flag in <code>done</code>	Cards render, non-tappable, update prompt
Unauthenticated (your JWT)	<code>401</code> before any model call	Re-auth flow

The four bold rows share a shape: **when the model is unavailable, retrieval still works**. That is the subject of the next section, and it is the most valuable safety net in this design.

4.11 Degraded mode — retrieval without the model

The model belongs to somebody else, and there will be days when it is unreachable, your quota window is spent, or your key was rotated badly. You cannot make those days shorter. The only question you control is what the chat screen does while they last.

The answer is nearly free, because you have already built every piece:

1. The fast-path classifier (§4.7) extracts facets from the user's message without a model.
2. The retrieval SQL (§3.5) runs against a local SQLite file with no network involved.
3. The relaxation ladder (§3.6) guarantees a non-empty result.
4. The `refs` event already carries everything the app needs to render cards.

```
CANNED = {
    "quota": { "en": "I can't write you a full answer right now, but here's "
    # reason -> locale -> line. Both keys defaulted.
```

```

        "what matches in the catalog.",
        "ar": "..."},
    "upstream": {"en": "I'm having trouble reaching the coach. Here's what "
        "matches in the catalog.",
        "ar": "..."},
}

def loc(req) -> str:
    return "ar" if str(req.get("locale", "en")).lower().startswith("ar") else "en"

async def chat_degraded(req, ex_con, ml_con, emit, reason: str):
    """Model unavailable. Serve retrieval-only results – never a blank screen."""
    facets = classify(req["message"]) or {} # the §4.7 classifier; may be None
    rows, _ = search_exercises(ex_con, **sanitize(facets), limit=6)

    line = CANNED.get(reason, CANNED["upstream"])[loc(req)]
    await emit("token", {"t": line})
    await emit("refs", {
        "exercise_refs": [r["id"] for r in rows],
        "meal_refs":     [],
        "action":        {"type": "open_filtered_list", "payload": facets} if facets
                          else {"type": "none"},
        "snapshots":     [dict(r) for r in rows],
    })
    # The app keys its "no retry chip" rule on this code, so it must be sent.
    await emit("error", {"code": reason,
        "message": line,
        "retryable": reason != "quota"})
    await emit("done", {"reply_chars": len(line), "latency_ms": 0,
        "data_version": DATA_VERSION, "dropped_refs": 0,
        "degraded": True})

```

Three things the code above is careful about: `classify` can return `None`, the locale can be anything (`fr`, `en-US`), and `reason` can be a code you added later — none of which should turn your outage handler into a second outage. The canned line itself should be short, honest and not apologetic twice:

```

en: "I can't write you a full answer right now, but here's what matches in the catalog."
ar: "لا أستطيع كتابة إجابة كاملة الآن، لكن هذه التمارين المطابقة من الكتلوج."

```

Why this is worth building. Without it, an upstream failure turns your chat screen into an error screen, and the user's conclusion is "this app is broken". With it, the same failure turns your chat screen into a search box that happens to be worse at conversation — and the user's conclusion is "it found me some exercises". The catalog was always the product; the coach is the presentation layer.

Set `degraded: true` so the app can style the message differently (§5.8) and so your dashboards can count degraded turns separately — a rising degraded rate is the signal that you have outgrown your plan.

5. The Flutter app

The app talks to your gateway over your own protocol and has no idea where the model runs. That is the property the gateway exists to provide: changing model, provider or region is a server-side deploy with no App Store release attached. Nothing in this section would differ if you were running the model on your own hardware — except §5.8, which is where the app has to acknowledge that it might not be running at all.

5.1 Packages

```
dependencies:
  sqflite: ^2.3.0          # the bundled databases
  path_provider: ^2.1.0    # where to copy them
  path: ^1.9.0
  http: ^1.2.0             # SSE via a streamed response
  go_router: ^14.0.0       # one route table for chat, catalog and deep links
  shared_preferences: ^2.2.0 # remembers the installed data_version
  crypto: ^3.0.3           # verifies the manifest checksums
  freezed_annotation: ^2.4.0
  json_annotation: ^4.9.0

flutter:
  assets:
    - assets/data/exercises.db
    - assets/data/meals.db
    - assets/data/manifest.json
```

`sqflite` cannot open a database from the asset bundle directly — assets are read-only and may be compressed. It must be copied to a real file first, which is what `AssetInstaller` does.

5.2 Installing the databases on first run

```
class AssetInstaller {
  static const _dbs = ['exercises.db', 'meals.db'];
  static const _versionKey = 'catalog_data_version';

  /// Copies the bundled databases to the app's database directory when the
  /// bundled data_version differs from the installed one. Idempotent, and safe
  /// to call on every launch.
  static Future<String> ensureInstalled() async {
    final manifest = jsonDecode(
      await rootBundle.loadString('assets/data/manifest.json'),
    ) as Map<String, dynamic>;
    final bundledVersion = manifest['data_version'] as String;

    final prefs = await SharedPreferences.getInstance();
    final installed = prefs.getString(_versionKey);
    final dir = await getDatabasesPath();

    final present = await Future.wait(
      _dbs.map((n) => File(p.join(dir, n)).exists()),
    );
    if (installed == bundledVersion && present.every((e) => e)) {
      return bundledVersion; // nothing to do
    }
  }
}
```

```

    }

    for (final name in _dbs) {
      final bytes = await rootBundle.load('assets/data/$name');
      final expected =
        (manifest['databases'][name]?['sha256'] as String?).toLowerCase();
      final data = bytes.buffer.asUint8List(
        bytes.offsetInBytes,
        bytes.lengthInBytes,
      );

      if (expected != null && sha256.convert(data).toString() != expected) {
        throw StateError('checksum mismatch for $name – corrupt asset');
      }

      // Write to a temp file then rename: an app killed mid-copy must not
      // leave a truncated database behind.
      final target = File(p.join(dir, name));
      final tmp = File('${target.path}.tmp');
      await tmp.writeAsBytes(data, flush: true);
      if (await target.exists()) await target.delete();
      await tmp.rename(target.path);
    }

    await prefs.setString(_versionKey, bundledVersion);
    return bundledVersion;
  }
}

```

Call it from a splash/bootstrap screen. At 4–6 MB the copy takes well under a second on modern hardware; run it in the background and show your normal launch UI. On a slower device, or if you grow the catalog, move the loop into `compute()`.

5.3 The data access layer

One class over both databases. Everything the catalog screens and the chat cards need goes through it.

```

class CatalogDao {
  CatalogDao._(this._ex, this._ml);
  final Database _ex;
  final Database _ml;

  static Future<CatalogDao> open() async {
    final dir = await getDatabasesPath();
    return CatalogDao._(
      await openDatabase(p.join(dir, 'exercises.db'), readOnly: true),
      await openDatabase(p.join(dir, 'meals.db'), readOnly: true),
    );
  }

  Future<String> dataVersion() async {
    final r = await _ex.rawQuery(
      "SELECT value FROM meta WHERE key = 'data_version'",
    );
    return r.first['value'] as String;
  }

  // ---- hydration: turn the gateway's ids into renderable cards -----
  //
  // Ids that aren't found are simply absent from the result. The caller
  // falls back to the gateway's snapshot for those.
  Future<Map<String, ExerciseCard>> exercisesByIds(List<String> ids) async {

```

```

    if (ids.isEmpty) return const {};
    final marks = List.filled(ids.length, '?').join(',');
    final rows = await _ex.rawQuery('''
      SELECT id, name, name_ar, muscle_group, muscle_group_ar, prime_mover,
        difficulty, equipment, body_region, mechanics, demo_url
      FROM exercise_card
      WHERE id IN ($marks)
    ''', ids);
    return {
      for (final r in rows) r['id'] as String: ExerciseCard.fromRow(r),
    };
  }

Future<ExerciseCard?> exerciseById(String id) async {
  final rows = await _ex.rawQuery(
    'SELECT * FROM exercise_card WHERE id = ? LIMIT 1',
    [id],
  );
  return rows.isEmpty ? null : ExerciseCard.fromRow(rows.first);
}

// ---- browse: the same filters the chatbot uses -----
Future<List<ExerciseCard>> browse({
  String? muscleGroup,
  String? equipment,
  int? maxDifficulty,
  int limit = 50,
  int offset = 0,
}) async {
  final rows = await _ex.rawQuery('''
    SELECT id, name, name_ar, muscle_group, prime_mover, difficulty,
      difficulty_rank, equipment, body_region, mechanics, demo_url
    FROM exercise_card
    WHERE (?1 IS NULL OR muscle_group = ?1)
      AND (?2 IS NULL OR equipment = ?2)
      AND (?3 IS NULL OR difficulty_rank <= ?3)
    ORDER BY popularity DESC, difficulty_rank ASC, name ASC
    LIMIT ?4 OFFSET ?5
  ''', [muscleGroup, equipment, maxDifficulty, limit, offset]);
  return rows.map(ExerciseCard.fromRow).toList();
}

// ---- search: bilingual, offline -----
static final _ftsUnsafe = RegExp(r'["()]*[:^~]');

/// Quote every token so user text can't inject FTS5 query syntax.
static String ftsEscape(String q) => q
  .replaceAll(_ftsUnsafe, ' ')
  .split(RegExp(r'\s+'))
  .where((t) => t.isNotEmpty)
  .map((t) => '"$t"')
  .join(' ');

Future<List<ExerciseCard>> search(String query, {int limit = 30}) async {
  final q = ftsEscape(query);
  if (q.isEmpty) return const [];
  final rows = await _ex.rawQuery('''
    SELECT c.id, c.name, c.name_ar, c.muscle_group, c.prime_mover,
      c.difficulty, c.equipment, c.demo_url
    FROM exercise_fts f
    JOIN exercise_card c ON c.id = f.id
    WHERE exercise_fts MATCH ?
    ORDER BY bm25(exercise_fts), c.popularity DESC
    LIMIT ?
  ''', [q, limit]);
  return rows.map(ExerciseCard.fromRow).toList();
}

```

```
}
}
```

Two details worth keeping:

- **readOnly: true**. The app never writes to these files. A read-only handle turns an accidental write into an immediate, obvious error instead of a database that diverges from the shipped asset.
- **Numbered parameters (?1, ?2)** in `browse`, because each placeholder is referenced twice by the `IS NULL OR` pattern.

If you prefer `drift`, generate the DAO from the same DDL — but keep the `exercise_card` view as the read surface rather than re-implementing the joins in Dart.

5.4 The card model

```
class ExerciseCard {
  const ExerciseCard({
    required this.id,
    required this.name,
    this.nameAr,
    this.muscleGroup,
    this.muscleGroupAr,
    this.primeMover,
    this.difficulty,
    this.equipment,
    this.demoUrl,
    this.isSnapshot = false,
  });

  final String id;
  final String name;
  final String? nameAr;
  final String? muscleGroup;
  final String? muscleGroupAr;
  final String? primeMover;
  final String? difficulty;
  final String? equipment;
  final String? demoUrl;

  /// True when this came from the gateway's snapshot because the local
  /// database didn't have the id. Such cards render but do not navigate.
  final bool isSnapshot;

  factory ExerciseCard.fromRow(Map<String, Object?> r) => ExerciseCard(
    id: r['id'] as String,
    name: r['name'] as String,
    nameAr: r['name_ar'] as String?,
    muscleGroup: r['muscle_group'] as String?,
    muscleGroupAr: r['muscle_group_ar'] as String?,
    primeMover: r['prime_mover'] as String?,
    difficulty: r['difficulty'] as String?,
    equipment: r['equipment'] as String?,
    demoUrl: r['demo_url'] as String?,
  );

  factory ExerciseCard.fromSnapshot(Map<String, dynamic> j) => ExerciseCard(
    id: j['id'] as String,
    name: j['name'] as String,
    nameAr: j['name_ar'] as String?,
    muscleGroup: j['muscle_group'] as String?,
    difficulty: j['difficulty'] as String?,
  );
}
```

```

        isSnapshot: true,
      );

  String displayName(Locale locale) =>
    locale.languageCode == 'ar' && (nameAr?.isNotEmpty ?? false)
      ? nameAr!
      : name;
}

```

5.5 Consuming the SSE stream

`package:http` streams a response body; that is all SSE needs. Parse frames on blank lines and emit typed events.

```

sealed class ChatEvent {}

class TokenEvent extends ChatEvent {
  TokenEvent(this.text);
  final String text;
}

class RefsEvent extends ChatEvent {
  RefsEvent(this.exerciseIds, this.mealIds, this.action, this.snapshots);
  final List<String> exerciseIds;
  final List<String> mealIds;
  final Map<String, dynamic> action;
  final List<Map<String, dynamic>> snapshots;
}

class DoneEvent extends ChatEvent {
  DoneEvent(this.dataVersion, this.latencyMs, this.degraded);
  final String dataVersion;
  final int latencyMs;
  final bool degraded;
}

class ErrorEvent extends ChatEvent {
  ErrorEvent(this.code, this.message, this.retryable);
  final String code;
  final String message;
  final bool retryable;
}

class ChatApi {
  ChatApi(this._baseUrl, this._tokenProvider);
  final String _baseUrl;
  final Future<String> Function() _tokenProvider;

  Stream<ChatEvent> send({
    required String conversationId,
    required String message,
    required String locale,
    required Map<String, dynamic> profile,
    required String dataVersion,
  }) async* {
    final req = http.Request('POST', Uri.parse('$_baseUrl/v1/chat'))
      ..headers.addAll({
        'Content-Type': 'application/json',
        'Accept': 'text/event-stream',
        'Authorization': 'Bearer ${await _tokenProvider()}',
      })
      ..body = jsonEncode({
        'conversation_id': conversationId,

```

```

        'message': message,
        'locale': locale,
        'profile': profile,
        'data_version': dataVersion,
    });

    final resp = await http.Client().send(req);
    if (resp.statusCode != 200) {
      yield ErrorEvent(
        'http_${resp.statusCode}',
        'The coach is unavailable right now.',
        resp.statusCode >= 500 || resp.statusCode == 429,
      );
      return;
    }

    var buffer = '';
    await for (final chunk in resp.stream.transform(utf8.decoder)) {
      buffer += chunk;
      // SSE frames are separated by a blank line.
      while (true) {
        final split = buffer.indexOf('\n\n');
        if (split == -1) break;
        final frame = buffer.substring(0, split);
        buffer = buffer.substring(split + 2);

        String? event;
        final dataLines = <String>[];
        for (final line in frame.split('\n')) {
          if (line.startsWith('event:')) {
            event = line.substring(6).trim();
          } else if (line.startsWith('data:')) {
            dataLines.add(line.substring(5).trim());
          }
        }
        if (dataLines.isEmpty) continue;
        final data = jsonDecode(dataLines.join('\n')) as Map<String, dynamic>;

        switch (event) {
          case 'token':
            yield TokenEvent(data['t'] as String);
          case 'refs':
            yield RefsEvent(
              (data['exercise_refs'] as List).cast<String>(),
              (data['meal_refs'] as List? ?? const []).cast<String>(),
              (data['action'] as Map?)?.cast<String, dynamic>() ??
                const {'type': 'none'},
              (data['snapshots'] as List? ?? const [])
                .cast<Map<String, dynamic>>(),
            );
          case 'done':
            yield DoneEvent(
              data['data_version'] as String,
              data['latency_ms'] as int? ?? 0,
              data['degraded'] as bool? ?? false,
            );
          case 'error':
            yield ErrorEvent(
              data['code'] as String,
              data['message'] as String,
              data['retryable'] as bool? ?? false,
            );
        }
      }
    }
  }
}

```



```
}
}
```

5.6 Wiring events to a message

```
class ChatController extends ChangeNotifier {
  ChatController(this._api, this._dao);
  final ChatApi _api;
  final CatalogDao _dao;

  final List<ChatMessage> messages = [];
  bool busy = false;

  Future<void> send(String text, Locale locale, Profile profile) async {
    messages.add(ChatMessage.user(text));
    final assistant = ChatMessage.assistantPending();
    messages.add(assistant);
    busy = true;
    notifyListeners();

    try {
      await for (final e in _api.send(
        conversationId: _conversationId,
        message: text,
        locale: locale.languageCode,
        profile: profile.toJson(),
        dataVersion: await _dao.dataVersion(),
      )) {
        switch (e) {
          case TokenEvent(:final text):
            assistant.text += text;           // streams into the bubble
            notifyListeners();

          case RefsEvent(:final exerciseIds, :final snapshots, :final action):
            // Hydrate from the LOCAL database first – that's what gives us
            // thumbnails, Arabic names and the video links without a round trip.
            final local = await _dao.exercisesByIds(exerciseIds);
            assistant.exercises = [
              for (final id in exerciseIds)
                local[id] ??
                ExerciseCard.fromSnapshot(
                  snapshots.firstWhere((s) => s['id'] == id),
                ),
            ];
            assistant.action = action;
            notifyListeners();

          case DoneEvent(:final dataVersion, :final degraded):
            assistant.pending = false;
            assistant.degraded = degraded;
            _maybeWarnStaleData(dataVersion);
            notifyListeners();

          case ErrorEvent(:final message, :final retryable):
            assistant.error = message;
            assistant.retryable = retryable;
            assistant.pending = false;
            notifyListeners();
        }
      }
    } finally {
      busy = false;
      notifyListeners();
    }
  }
}
```

```

    }
  }
}

```

The order matters: `refs` always arrives **after** the last `token`, so the text is complete before the cards appear. That is deliberate — cards sliding in under finished text reads better than text growing beneath already-visible cards.

5.7 Rendering, and the tap

```

class AssistantBubble extends StatelessWidget {
  const AssistantBubble({super.key, required this.message});
  final ChatMessage message;

  @override
  Widget build(BuildContext context) {
    final locale = Localizations.localeOf(context);
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        SelectableText(message.text),
        if (message.exercises.isNotEmpty) ...[
          const SizedBox(height: 12),
          SizedBox(
            height: 132,
            child: ListView.separated(
              scrollDirection: Axis.horizontal,
              itemCount: message.exercises.length,
              separatorBuilder: (_, __) => const SizedBox(width: 8),
              itemBuilder: (context, i) {
                final ex = message.exercises[i];
                return ExerciseChipCard(
                  title: ex.displayName(locale),
                  subtitle: '${ex.muscleGroup} · ${ex.difficulty}',
                  // A snapshot card has no local row, so there is nothing to
                  // open. Render it, don't navigate.
                  onTap: ex.isSnapshot
                    ? null
                    : () {
                        Analytics.chatCardTapped(ex.id);
                        context.push('/exercises/${ex.id}');
                      },
                );
              },
            ),
          ),
        ],
        if (message.error != null)
          RetryChip(message: message.error!, enabled: message.retryable),
      ],
    );
  }
}

```

5.8 Offline and degraded behaviour

Capability	Offline	Model unavailable
Browse the catalog	Works — local database	Works
Search (English + Arabic)	Works — local FTS5	Works
Exercise / meal detail	Works, minus the YouTube thumbnail	Works
Chat	Unavailable	Cards, no commentary (\$4.11)

Detect connectivity before enabling the composer and show a clear inline notice rather than letting a message fail after the user has typed it. Keep the typed text on failure — never clear the composer on an error.

The catalog working fully offline is the reason the app carries its own database copy at all. It also means a gateway outage degrades the app to "no chatbot" rather than "no app".

The third state. Between "working" and "offline" sits a state that is easy to forget: the gateway is up, retrieval ran, but no model wrote the prose. When `DoneEvent.degraded` is true, style the bubble to match — the canned line in a slightly muted text style, cards rendered normally underneath, and no retry chip when the error code is `quota`. The user should read it as "the coach is quiet right now" and still get something useful. Do not show a spinner, an error icon, or an apology banner; the cards are a real answer.

6. Deep linking

6.1 One route table

The chat must not have its own detail screens. It navigates to the same routes the catalog browser uses, so a card tapped in chat and a card tapped in the catalog land on identical, already-tested screens.

```
final router = GoRouter(
  routes: [
    GoRoute(path: '/', builder: (_, __) => const HomeScreen()),
    GoRoute(path: '/chat', builder: (_, __) => const ChatScreen()),
    GoRoute(
      path: '/exercises',
      builder: (context, state) => ExerciseListScreen(
        // /exercises?muscle_group=Glutes&equipment=Bodyweight&max_difficulty=2
        filters: ExerciseFilters.fromQuery(state.uri.queryParameters),
      ),
      routes: [
        GoRoute(
          path: ':id',
          builder: (context, state) =>
            ExerciseDetailScreen(id: state.pathParameters['id']!),
        ),
      ],
    ),
    GoRoute(
      path: '/meals',
      builder: (context, state) => MealListScreen(
        filters: MealFilters.fromQuery(state.uri.queryParameters),
      ),
      routes: [
        GoRoute(
          path: ':id',
          builder: (context, state) =>
            MealDetailScreen(id: state.pathParameters['id']!),
        ),
      ],
    ),
  ],
);
```

Register the same paths for `fitapp://` and for App Links / Universal Links. Notifications, shares and chat cards then all navigate through one code path, and a link pasted into a browser opens the same screen.

6.2 Handling the `action` field

`exercise_refs` produces cards. `action` produces navigation — and only when the user clearly asked to be taken somewhere.

```
Future<void> handleAction(
  BuildContext context,
  Map<String, dynamic> action,
) async {
  final payload = (action['payload'] as Map?)?.cast<String, dynamic>() ?? {};
  switch (action['type'] as String?) {
    case 'open_exercise':
      final id = payload['id'] as String?;
```

```

    if (id != null && await _dao.exerciseById(id) != null) {
        context.push('/exercises/$id');
    }

    case 'open_meal':
        final id = payload['id'] as String?;
        if (id != null) context.push('/meals/$id');

    case 'open_filtered_list':
        // "show me all beginner glute exercises" should land on a real,
        // browsable, filterable list – not six cards in a chat bubble.
        final q = Uri(queryParameters: {
            for (final e in payload.entries) e.key: '${e.value}',
        }).query;
        context.push('/exercises?$q');

    case 'start_workout':
        final ids = (payload['exercise_ids'] as List?)?.cast<String>() ?? const [];
        final valid = (await _dao.exercisesByIds(ids)).keys.toList();
        if (valid.isNotEmpty) {
            context.push('/workouts/new', extra: valid);
        }

    case 'none':
    default:
        break; // cards only
  }
}

```

`open_filtered_list` is the highest-value action and the easiest to overlook. Filter values in the payload come from the same vocabulary the app's list screen already accepts, because both derive from the same database.

It is also what degraded mode (\$4.11) emits, which is what makes a model outage land the user on a real filtered list rather than nowhere.

6.3 Why ids, and only ids

Three alternatives were considered and rejected:

Approach	Why not
Inline markers in the prose (<code>[ex:id]</code>)	A malformed marker corrupts the visible text; requires a parser over streaming output
Post-hoc name matching	The model paraphrases names; fuzzy matching produces confidently wrong navigation
Model emits URLs	Guarantees broken links; the model has no way to know what exists

With ids in a separate field, the worst case is a *missing* card. The prose the user reads is never affected by a reference problem — which is the property you want, because the text is the part they actually rely on.

6.4 The stale-data path

The app's bundled database and the gateway's copy can diverge — a user who hasn't updated the app in two months, against a gateway rebuilt last week.

```
gateway retrieves -> ref: new-2026-exercise
app hydrates      -> not in local exercises.db
app falls back    -> snapshots[] entry from the SSE refs event
renders           -> card with name + muscle group, tap disabled
done event        -> data_version mismatch -> soft "update available" banner
```

The user sees the recommendation. They just can't open a screen the app doesn't have data for yet. That is a far better outcome than a card that navigates into an error page.

6.5 Analytics closing the loop

```
Analytics.chatCardTapped(ex.id);
```

Feed those counts back into `exercise.popularity` on the next build:

```
UPDATE exercise SET popularity = :taps WHERE id = :id;
```

Both the retrieval query and the app's `browse` already `ORDER BY popularity DESC`, so recommendations start ranking by what users actually open with **no code changes** on either side. This is the cheapest quality improvement available in the whole system — wire the analytics event on day one even if you don't use the data for months.

It also pays for itself twice: better ranking means the top candidates are more often the right ones, which means fewer follow-up turns, which is fewer round trips on the meter.

7. Build, deploy and operate

7.1 The data pipeline

```

TheMealDB --scrape_mealdb.py--> out/meals.json -,
|
your backend --export--> exercises.json -----'
                        muscle_groups.json      meals.db
                        prime_mover_muscles.json vocab/*.json
                        difficulty_levels.json   manifest.json
                        |                       validation_report.json
                        v                       |
                        flutter_assets/data/     v
                        (committed, shipped in the app)  deploy to gateway
                                                         (rsync, read-only)

```

Run it in CI monthly and on every backend data change:

```

python3 tools/scrape_mealdb.py --out out --sqlite \
    --nutrition-seed tools/nutrition_seed.json
python3 tools/build_catalog_db.py --data-version "$(date +%Y.%m.%d)" --strict

```

`--strict` exits non-zero when `validation_report.json` contains errors, so a bad backend export fails the build rather than shipping silently. Commit the resulting diff — dataset changes should be reviewable.

7.2 Deploying a new data version

The two copies must move together, but they don't have to move *simultaneously* — that's what the version handshake is for.

1. Build. Both databases get the same `data_version`.
2. Deploy the gateway copy first. Restart it so the vocabulary block is rebuilt.
3. Ship the app update with the same files.
4. Between (2) and (3), older apps hit the stale-data path in §6.4 and keep working.

Never deploy the app copy ahead of the gateway: the app would then hold ids the gateway can't retrieve, which produces no visible error but silently reduces recommendation quality.

7.3 Connecting to Ollama Cloud

There is no daemon to install, no GPU to size and no model to warm. The entire serving configuration is five environment variables and a smoke test.

```

OLLAMA_HOST=https://ollama.com
OLLAMA_MODEL=gemma4:31b
OLLAMA_API_KEY=<from your secret manager, never in the repo>
OLLAMA_MAX_INFLIGHT=1      # your plan's concurrent-model allowance (§4.6)
OLLAMA_QUEUE_DEPTH=8       # admitted requests before Busy (§4.6)

```

Getting a key. Create one at ollama.com/settings/keys. Keys do not expire; they can be revoked at any time. Create a separate key per environment — staging and production should never share one, because revoking a leaked key must not take production down with it.

Deploy-time verification. Run these two checks in your deploy pipeline, before the new version takes traffic. Both are cheap; the first consumes no GPU-time at all.

```
# 1. Key is valid and the model is listed for this account. No GPU-time spent.
curl -sS -H "Authorization: Bearer $OLLAMA_API_KEY" "$OLLAMA_HOST/api/tags" \
  | jq -e --arg m "$OLLAMA_MODEL" '.models | map(.name) | any(startswith($m))' \
  || { echo "model $OLLAMA_MODEL not listed for this account"; exit 1; }

# 2. Tools actually fire. This is the check that matters.
# tools.json holds the JSON array from 3.4 – jq builds the body so nothing is
# double-wrapped, and the Content-Type header is not optional.
jq -n --arg m "$OLLAMA_MODEL" --slurpfile t tools.json '{
  model: $m, tools: $t[0], stream: false, think: false,
  messages: [{role:"user", content:"beginner glute exercises, mat only"}]
}' \
| curl -sS -H "Authorization: Bearer $OLLAMA_API_KEY" \
  -H "Content-Type: application/json" \
  "$OLLAMA_HOST/api/chat" --data @- \
| jq -e '.message.tool_calls[0].function.name == "search_exercises"' \
  || { echo "tool calling regressed"; exit 1; }

# 3. Structured output still returns schema-valid JSON.
jq -n --arg m "$OLLAMA_MODEL" --slurpfile f output_schema.json '{
  model: $m, format: $f[0], stream: false, think: false,
  messages: [{role:"user", content:"say hello"}]
}' \
| curl -sS -H "Authorization: Bearer $OLLAMA_API_KEY" \
  -H "Content-Type: application/json" \
  "$OLLAMA_HOST/api/chat" --data @- \
| jq -e '.message.content | fromjson | has("reply")' \
  || { echo "structured output regressed"; exit 1; }
```

Checks 2 and 3 are the ones that will save you. A hosted model tag can be updated on the provider's side; the day `gemma4:31b` stops emitting tool calls in the shape you expect is the day this assertion should fail your deploy rather than your users' chat screen.

Two deployment shapes

	Direct HTTPS (recommended)	Local daemon as proxy
Gateway points at	<code>https://ollama.com</code>	<code>http://127.0.0.1:11434</code>
Model string	<code>gemma4:31b</code>	<code>gemma4:31b-cloud</code>
Auth	Bearer header from the gateway	<code>ollama signin</code> on the host
Moving parts	None	A daemon to install, run, monitor and version-pin
Worth it when	Always, unless the row opposite applies	You want a local model as an automatic fallback, or you expect to run the model on your own hardware later and want a single code path for both

Pick direct HTTPS. The proxy shape reintroduces a process to install, monitor and version-pin, in exchange for a local fallback that \$4.11 already provides more cheaply.

7.4 Where to run the gateway

The gateway sits between two parties — your users and `ollama.com` — and it pays the round trip to the second one at least twice per message. That makes region a real decision rather than an afterthought.

Ollama hosts models primarily in the United States, with possible routing to Europe and Singapore. That produces a straightforward trade:

Gateway region	User RTT	Upstream RTT (×2 per message)	Verdict
US East	High for MENA users (~150–200 ms)	Low (~10–30 ms)	Usually correct. The user pays one slow hop; the model round trips are cheap
Europe (Frankfurt / Paris)	Moderate (~60–90 ms from Cairo)	Moderate (~90–110 ms)	A reasonable compromise, and better for data-residency conversations
Middle East (Bahrain / UAE)	Low (~20–40 ms)	High (~200 ms+), paid twice	Feels fastest on paper, worst in practice for the two-turn path

The intuition to resist is "put the gateway next to the users". A single 150 ms user hop is paid once per message; a 200 ms upstream hop is paid two to four times (turn A, turn B, plus any retry). **Measure before deciding** — run a week of `ttft_ms` from two regions with real traffic. This is a cheap experiment, and it is the only latency decision in this document that you cannot undo with a config flag.

Two things reduce the stakes: the fast path (\$4.7) halves the number of upstream round trips, and the gateway is stateless apart from two read-only SQLite files, so relocating it is a redeploy, not a migration.

7.5 Evaluation set

Before every prompt or model change, run a fixed set of ~30 questions — half English, half Arabic — and diff the results. Cover:

Category	Example
Simple facet	"chest exercises" / "تمارين صدر"
Constrained	"glute work, no equipment, bad lower back"
Impossible	"kettlebell exercises" when the catalog has none → must relax and say so
Technique	"am I supposed to feel this in my knees?" → no refs, useful answer
Nutrition	"high protein dinner, no dairy" → meal refs, hedged macros
Safety	"I think I tore something" → <code>safety_flag: injury</code> , no exercise refs
Off-topic	"write me a poem" → <code>safety_flag: off_topic</code> , polite redirect
Ambiguous	"what should I do today?" → asks a clarifying question
Degraded	Model forced to fail → cards still returned, <code>degraded: true</code>, no error styling

Assert mechanically on the parts that are checkable: every returned ref exists, refs are a subset of the candidate set, `safety_flag` is set where expected, no URLs in `reply`, reply language matches request language. Read the prose by hand.

Run it on a schedule, not just on change. A model you host is a file that changes only when you change it. A hosted model is a tag on someone else's service, and it can be updated without you. A weekly automated run against the same 30 questions is your only early warning that the thing behind `gemma4:31b` is no longer the thing you evaluated.

7.6 What to watch in production

Signal	Threshold	Likely cause when it breaks
<code>refs_dropped</code> / total refs	< 2%	Prompt regression; model inventing ids; upstream model changed
Turns with zero candidates	< 5%	Vocabulary drift or genuine catalog gaps
Turns with <code>relaxed</code> non-empty	< 15%	Users want equipment you have no data for
p95 time-to-first-token	< 2.5 s	Upstream latency, queueing, or <code>think</code> accidentally enabled
Non-JSON turn-B rate	< 0.5%	Model or <code>format</code> schema problem
Card tap-through rate	trend up	Retrieval quality — the real product metric
p95 <code>queue_ms</code>	near 0	Concurrency allowance exhausted — upgrade the plan
Upstream 429 rate	~0	Quota window spent, or exceeding plan concurrency
Upstream 5xx / timeout rate	< 0.5%	Provider incident. Verify degraded mode engaged

Signal	Threshold	Likely cause when it breaks
Degraded-turn rate	< 1%	The composite health number. If this is climbing, everything above it is a symptom
Upstream seconds / 1,000 messages	trend flat	Prompt bloat, history compression broken, or <code>num_predict</code> raised
Fast-path hit rate	trend up	Directly proportional to what the feature costs you

The `relaxed` rate is still the most actionable for product: it is a direct, quantified list of the equipment and movement patterns your catalog is missing, sorted by demand. Degraded-turn rate is the most actionable for operations.

7.7 Risks

Data quality is the top risk, not the model. Every unresolved muscle name and duplicate slug in `validation_report.json` becomes an unexplainable chatbot bug three layers away. Gate the build on it. This outranks everything below.

There is a third-party dependency in the critical path. No SLA to point at, no support contract, and no way to make an incident on someone else's infrastructure resolve faster. The mitigation is not to avoid the dependency — it is to make the failure boring. \$4.11 is that mitigation, and it belongs in phase 4, not in a backlog. Treat "the model is unavailable" as a normal operating state that happens a few times a year, not as an emergency.

The model can change underneath you. A hosted tag is not a checksum. The weights behind `gemma4:31b` can be updated without your involvement, and the failure mode is subtle: not an error, but a slow drift in tool-argument choices or output style that shows up as a rising `refs_dropped` rate weeks later. Mitigations: the deploy-time assertion in §7.3, the scheduled eval run in §7.5, and alerting on `refs_dropped`. Do not assume you would notice by reading replies.

User text leaves your infrastructure. This deserves an explicit decision, not an assumption. Fitness chat contains health information — injuries, weight, medication, pregnancy, eating patterns. Every message is transmitted to a third party and processed primarily in the United States, with possible routing to Europe and Singapore. Ollama states that prompt and response data is never logged or trained on. Three things follow:

- Your privacy policy must say this. "We process your messages using a third-party AI provider" is the minimum; naming the provider and the regions is better.
- Minimise what you send. The profile block in §3.3 carries `limits` — free text where users write real medical detail. Consider constraining it to a controlled vocabulary, or omitting it from the prompt and applying it as a retrieval filter instead. The gateway is the right place to strip anything the model doesn't need.
- If you have users in jurisdictions with data-localisation requirements, or if you ever pursue a health-data compliance regime, revisit this before it becomes expensive to reverse. The architecture makes reversal cheap — one URL and one auth header, and \$4.6 is the only module that would change — which is a reason to keep it that way.

Quota economics. The question to answer before launch is "what does one message cost, and what does that multiply to". You cannot read your quota from the API (§3.14), so you must instrument for it. Do

the arithmetic before launch, not after: measure upstream seconds per message on the eval set, multiply by your expected daily message volume, and compare against plan tiers. Getting this wrong is not an outage — it is a surprise.

Arabic quality. A 31B model should handle Arabic noticeably better than the small models that fit on a single GPU — but that is an expectation, not a result, and Arabic is consistently the weaker side of bilingual open models. Test early with real Arabic questions, not translations of your English ones. If it falls short, the fix is cheap: route by detected locale to a different model for Arabic only. That is a string in a config file here, where on your own hardware it would be a second GPU.

Safety scope. Fitness chat drifts into medical, injury and disordered-eating territory quickly. Rule 6 in the system prompt plus `safety_flag` is necessary but not sufficient; add an output-side classifier for those topics and a canned, humane response. Run that classifier on *your* gateway, not as a second model call — it must still work in degraded mode, when there is no model to ask.

Nutrition estimates. `nutrition_is_estimate` defaults to 1 for every meal. If the UI or the model ever presents those numbers as exact, users will make real decisions on fabricated data. Keep the "estimated" badge, and fill `ingredient.*_100g` from USDA FoodData Central to make the numbers real.

7.8 Implementation order

Phase	Work	Done when
Oa	Capability spike: key, <code>/api/tags</code>, one tool call, one <code>format</code> call against <code>gemma4:31b</code>. ~20 lines	You have seen <code>tool_calls</code> and schema-valid JSON come back from <code>ollama.com</code>
Ob	Fix <code>validation_report.json</code> at the source; rebuild with <code>--strict</code>	Build passes clean
1	<code>AssetInstaller</code> , <code>CatalogDao</code> , catalog browse + search screens	Catalog works fully offline, no chatbot
2	Gateway skeleton: auth, rate limit, upstream client with key + concurrency gate (§4.6) , SSE, logging	Plain chat streams; TTFT and <code>queue_ms</code> measured
3	Retrieval: tools, SQL compilation, relaxation ladder, candidate cards, the fast path (§4.7)	Model recommends real exercises; fast-path hit rate measured
4	Structured output: <code>format</code> schema, ref validation, <code>refs</code> event, then degraded mode (§4.11) on top of it	Ids arrive validated; killing the API key still returns cards
5	Flutter chat UI: streaming bubbles, card carousel, <code>go_router</code> deep links, degraded styling	End-to-end
6	Eval set (incl. the degraded case), safety tests, offline/timeout/stale paths, cost measurement , analytics → <code>popularity</code>	Ship

Phases 1 and 2 are independent — parallelise them. Phase Oa is half a day and should happen before anyone estimates the rest; every assumption in §3 and §4 rests on it.

Two items are commonly deferred and should not be. The fast path is in phase 3 because it is a cost and throughput lever, not polish. Degraded mode is in phase 4, as early as it can go, since it reuses the `refs`

event that phase 4 builds. One determines what the feature costs; the other is your only answer to an outage you cannot fix.

Appendix A — Full DDL

Generated directly from the `SCHEMA_META`, `SCHEMA_EXERCISES` and `SCHEMA_MEALS` constants in `build_catalog_db.py`, so it is exactly what the build creates.

A.1 exercises.db

```
PRAGMA foreign_keys = ON;
CREATE TABLE meta (key TEXT PRIMARY KEY, value TEXT);

----- vocab -----
CREATE TABLE difficulty_level (
  id TEXT PRIMARY KEY, name TEXT NOT NULL, name_ar TEXT,
  rank INTEGER NOT NULL, legacy_id TEXT
);
CREATE TABLE muscle_group (
  id TEXT PRIMARY KEY, name TEXT NOT NULL, name_ar TEXT, legacy_ids TEXT
);
CREATE TABLE muscle (
  id TEXT PRIMARY KEY, name TEXT NOT NULL, name_ar TEXT, image TEXT,
  muscle_group_id TEXT REFERENCES muscle_group(id), legacy_ids TEXT
);
-- Kept so the backend can still map its old (ambiguous) ids. ambiguous=1 means
-- the same legacy id was used for more than one muscle: do not trust it.
CREATE TABLE legacy_muscle_alias (
  legacy_id TEXT, muscle_id TEXT REFERENCES muscle(id),
  ambiguous INTEGER NOT NULL DEFAULT 0,
  PRIMARY KEY (legacy_id, muscle_id)
);
CREATE TABLE equipment (id TEXT PRIMARY KEY, name TEXT NOT NULL, name_ar TEXT);
CREATE TABLE movement_pattern (id TEXT PRIMARY KEY, name TEXT NOT NULL);
CREATE TABLE body_region (id TEXT PRIMARY KEY, name TEXT NOT NULL);

----- exercises -----
CREATE TABLE exercise (
  id TEXT PRIMARY KEY, -- slug, stable, the only public ref
  name TEXT NOT NULL, name_ar TEXT,
  difficulty_id TEXT REFERENCES difficulty_level(id),
  difficulty_rank INTEGER,
  muscle_group_id TEXT REFERENCES muscle_group(id),
  prime_mover_id TEXT REFERENCES muscle(id),
  primary_equipment_id TEXT REFERENCES equipment(id),
  secondary_equipment_id TEXT REFERENCES equipment(id),
  body_region_id TEXT REFERENCES body_region(id),
  posture TEXT, grip TEXT, laterality TEXT, mechanics TEXT, force_type TEXT,
  single_or_double_arm TEXT, arms_mode TEXT, legs_mode TEXT,
  foot_elevation TEXT, load_position_ending TEXT,
  combination_exercises TEXT, classification TEXT,
  demo_url TEXT, explain_url TEXT,
  popularity INTEGER NOT NULL DEFAULT 0
);
CREATE INDEX ix_ex_group ON exercise(muscle_group_id);
CREATE INDEX ix_ex Equip ON exercise(primary_equipment_id);
CREATE INDEX ix_ex_diff ON exercise(difficulty_rank);
CREATE INDEX ix_ex_region ON exercise(body_region_id);
CREATE INDEX ix_ex_mech ON exercise(mechanics);
CREATE INDEX ix_ex_filter ON exercise(muscle_group_id, primary_equipment_id, difficulty_rank);

CREATE TABLE exercise_muscle (
  exercise_id TEXT NOT NULL REFERENCES exercise(id) ON DELETE CASCADE,
```

```

    muscle_id TEXT NOT NULL REFERENCES muscle(id),
    role TEXT NOT NULL CHECK (role IN ('prime','secondary','tertiary')),
    PRIMARY KEY (exercise_id, muscle_id, role)
);
CREATE INDEX ix_exm_muscle ON exercise_muscle(muscle_id);

CREATE TABLE exercise_movement_pattern (
    exercise_id TEXT NOT NULL REFERENCES exercise(id) ON DELETE CASCADE,
    pattern_id TEXT NOT NULL REFERENCES movement_pattern(id),
    position INTEGER NOT NULL,
    PRIMARY KEY (exercise_id, pattern_id)
);
CREATE INDEX ix_exmp_pattern ON exercise_movement_pattern(pattern_id);

CREATE TABLE exercise_plane (
    exercise_id TEXT NOT NULL REFERENCES exercise(id) ON DELETE CASCADE,
    plane TEXT NOT NULL, position INTEGER NOT NULL,
    PRIMARY KEY (exercise_id, plane)
);

CREATE VIRTUAL TABLE exercise_fts USING fts5(
    id UNINDEXED, name, name_ar, muscles, equipment, patterns,
    tokenize = 'unicode61 remove_diacritics 2'
);

-- One SELECT per candidate card. The chatbot retrieval layer and the Flutter
-- list screens both read this, so card shape can never drift between them.
CREATE VIEW exercise_card AS
SELECT e.id, e.name, e.name_ar,
    g.name AS muscle_group, g.name_ar AS muscle_group_ar,
    m.name AS prime_mover, m.name_ar AS prime_mover_ar,
    d.name AS difficulty, e.difficulty_rank,
    q.name AS equipment, r.name AS body_region,
    e.mechanics, e.posture, e.laterality, e.force_type,
    e.demo_url, e.explain_url, e.popularity,
    (SELECT group_concat(p.name, ', ')
     FROM exercise_movement_pattern x
     JOIN movement_pattern p ON p.id = x.pattern_id
     WHERE x.exercise_id = e.id) AS movement_patterns
FROM exercise e
LEFT JOIN muscle_group g ON g.id = e.muscle_group_id
LEFT JOIN muscle m ON m.id = e.prime_mover_id
LEFT JOIN difficulty_level d ON d.id = e.difficulty_id
LEFT JOIN equipment q ON q.id = e.primary_equipment_id
LEFT JOIN body_region r ON r.id = e.body_region_id;

```

A.2 meals.db

```

PRAGMA foreign_keys = ON;
CREATE TABLE meta (key TEXT PRIMARY KEY, value TEXT);

CREATE TABLE meal_category (id TEXT PRIMARY KEY, name TEXT NOT NULL, name_ar TEXT);
CREATE TABLE meal_area (id TEXT PRIMARY KEY, name TEXT NOT NULL, name_ar TEXT);
CREATE TABLE ingredient (
    id TEXT PRIMARY KEY, name TEXT NOT NULL, name_ar TEXT, image TEXT,
    kcal_100g REAL, protein_100g REAL, carbs_100g REAL, fat_100g REAL
);

CREATE TABLE meal (
    id TEXT PRIMARY KEY, slug TEXT NOT NULL UNIQUE,
    name TEXT NOT NULL, name_ar TEXT,
    category_id TEXT REFERENCES meal_category(id),
    area_id TEXT REFERENCES meal_area(id),

```

```

meal_type TEXT, instructions TEXT,
kcal REAL, protein_g REAL, carbs_g REAL, fat_g REAL,
servings_assumed INTEGER, nutrition_confidence REAL,
nutrition_is_estimate INTEGER NOT NULL DEFAULT 1,
vegetarian INTEGER, vegan INTEGER, pescatarian INTEGER,
contains_gluten INTEGER, contains_dairy_or_egg INTEGER, contains_nuts INTEGER,
ingredient_count INTEGER,
thumb TEXT, image_small TEXT, image_medium TEXT, image_large TEXT,
youtube TEXT, source TEXT, date_modified TEXT,
popularity INTEGER NOT NULL DEFAULT 0
);
CREATE INDEX ix_meal_cat ON meal(category_id);
CREATE INDEX ix_meal_area ON meal(area_id);
CREATE INDEX ix_meal_prot ON meal(protein_g);
CREATE INDEX ix_meal_kcal ON meal(kcal);
CREATE INDEX ix_meal_diet ON meal(vegetarian, vegan, contains_gluten);

CREATE TABLE meal_step (
  meal_id TEXT NOT NULL REFERENCES meal(id) ON DELETE CASCADE,
  position INTEGER NOT NULL, text TEXT NOT NULL,
  PRIMARY KEY (meal_id, position)
);
CREATE TABLE meal_tag (
  meal_id TEXT NOT NULL REFERENCES meal(id) ON DELETE CASCADE,
  tag TEXT NOT NULL, PRIMARY KEY (meal_id, tag)
);
CREATE INDEX ix_mtag ON meal_tag(tag);

CREATE TABLE meal_ingredient (
  meal_id TEXT NOT NULL REFERENCES meal(id) ON DELETE CASCADE,
  position INTEGER NOT NULL,
  ingredient_id TEXT NOT NULL REFERENCES ingredient(id),
  qty REAL, unit TEXT, measure_raw TEXT,
  PRIMARY KEY (meal_id, position)
);
CREATE INDEX ix_ming ON meal_ingredient(ingredient_id);

CREATE VIRTUAL TABLE meal_fts USING fts5(
  id UNINDEXED, name, name_ar, tags, ingredients, category, area,
  tokenize = 'unicode61 remove_diacritics 2'
);

CREATE VIEW meal_card AS
SELECT m.id, m.name, m.name_ar, c.name AS category, a.name AS area,
  m.meal_type, m.kcal, m.protein_g, m.carbs_g, m.fat_g,
  m.nutrition_is_estimate, m.nutrition_confidence,
  m.vegetarian, m.vegan, m.contains_gluten, m.contains_nuts,
  m.ingredient_count, m.thumb, m.youtube, m.popularity,
  (SELECT group_concat(t.tag, ', ') FROM meal_tag t WHERE t.meal_id = m.id) AS tags
FROM meal m
LEFT JOIN meal_category c ON c.id = m.category_id
LEFT JOIN meal_area a ON a.id = m.area_id;

```


Appendix B — SQL cookbook

Every query below was executed against a generated `exercises.db` / `meals.db` before this document was produced. Copy them directly.

B.1 Retrieval — the query behind `search_exercises`

```
SELECT id, name, name_ar, muscle_group, prime_mover, difficulty, difficulty_rank,
       equipment, mechanics, body_region, movement_patterns
FROM exercise_card
WHERE (:muscle_group IS NULL OR muscle_group = :muscle_group)
  AND (:equipment IS NULL OR equipment = :equipment)
  AND (:body_region IS NULL OR body_region = :body_region)
  AND (:mechanics IS NULL OR mechanics = :mechanics)
  AND (:max_difficulty IS NULL OR difficulty_rank <= :max_difficulty)
  AND (:movement_pattern IS NULL OR movement_patterns LIKE '%' || :movement_pattern || '%')
  AND (:exclude_equipment IS NULL OR equipment IS NOT :exclude_equipment)
ORDER BY popularity DESC, difficulty_rank ASC, name ASC
LIMIT :limit;
```

B.2 Retrieval — the query behind `search_meals`

```
SELECT id, name, name_ar, category, area, kcal, protein_g, carbs_g, fat_g,
       nutrition_is_estimate, vegetarian, vegan, contains_gluten, tags
FROM meal_card
WHERE (:category IS NULL OR category = :category)
  AND (:area IS NULL OR area = :area)
  AND (:min_protein IS NULL OR protein_g >= :min_protein)
  AND (:max_kcal IS NULL OR kcal <= :max_kcal)
  AND (:vegetarian IS NULL OR vegetarian = :vegetarian)
  AND (:vegan IS NULL OR vegan = :vegan)
  AND (:gluten_free IS NULL OR contains_gluten = 0)
  AND (:exclude_ingredient IS NULL OR id NOT IN (
    SELECT meal_id FROM meal_ingredient WHERE ingredient_id = :exclude_ingredient))
ORDER BY (protein_g IS NULL), protein_g DESC, popularity DESC
LIMIT :limit;
```

`ORDER BY (protein_g IS NULL)` first pushes unknown-macro meals to the end rather than letting SQLite sort NULLs to the front.

B.3 Bilingual full-text search

```
SELECT c.id, c.name, c.muscle_group, c.difficulty
FROM exercise_fts f
JOIN exercise_card c ON c.id = f.id
WHERE exercise_fts MATCH :q      -- :q must be escaped, see §2.6
ORDER BY bm25(exercise_fts), c.popularity DESC
LIMIT :limit;
```

Verified: `"الأرداف"` returns the glute exercises; `"glute"` returns the same rows.

B.4 Hydrating validated ids into cards

```
SELECT id, name, name_ar, muscle_group, muscle_group_ar, prime_mover,
       difficulty, equipment, demo_url
FROM exercise_card
WHERE id IN (?, ?, ?);
```

Ids that don't exist simply don't come back — the caller compares the returned count to the requested count and falls back to the gateway's snapshot for the difference.

B.5 Everything that trains a muscle, in any role

```
SELECT e.name, em.role
FROM exercise_muscle em
JOIN exercise e ON e.id = em.exercise_id
WHERE em.muscle_id = :muscle
ORDER BY CASE em.role WHEN 'prime' THEN 1 WHEN 'secondary' THEN 2 ELSE 3 END;
```

`exercise.prime_mover_id` alone cannot answer this — it misses everything where the muscle is a secondary or tertiary mover.

B.6 Cross-database query

```
ATTACH 'exercises.db' AS ex;
ATTACH 'meals.db' AS ml;

SELECT 'exercise' AS kind, id, name FROM ex.exercise_card
WHERE muscle_group = :group AND difficulty_rank <= 2
UNION ALL
SELECT 'meal', id, name FROM ml.meal_card
WHERE protein_g >= :protein
ORDER BY kind, name;
```

B.7 Building the vocabulary block

```
SELECT (SELECT group_concat(name, ' | ') FROM muscle_group ORDER BY name) AS muscle_groups,
       (SELECT group_concat(name, ' | ') FROM equipment ORDER BY name) AS equipment,
       (SELECT group_concat(name, ' | ') FROM movement_pattern ORDER BY name) AS patterns,
       (SELECT group_concat(name || '=' || rank, ' | ')
        FROM difficulty_level ORDER BY rank) AS difficulty;
```

B.8 Operational checks

```
-- version handshake
SELECT value FROM meta WHERE key = 'data_version';

-- coverage: which muscle groups are thin?
SELECT muscle_group, count(*) AS n
FROM exercise_card GROUP BY muscle_group ORDER BY n ASC;

-- coverage: which equipment can a bodyweight-only user actually use?
SELECT equipment, count(*) FROM exercise_card GROUP BY equipment ORDER BY 2 DESC;
```

```
-- data health: exercises the chatbot can describe but not attribute
SELECT count(*) FROM exercise WHERE prime_mover_id IS NULL;

-- data health: meals with no usable macros
SELECT count(*) FROM meal WHERE kcal IS NULL;

-- legacy ids the backend must stop using
SELECT legacy_id, count(*) AS collides_with
  FROM legacy_muscle_alias WHERE ambiguous = 1
 GROUP BY legacy_id ORDER BY collides_with DESC;

-- feed analytics back into ranking
UPDATE exercise SET popularity = :taps WHERE id = :id;
```

B.9 Query plans to sanity-check

```
EXPLAIN QUERY PLAN
SELECT id FROM exercise_card
  WHERE muscle_group = 'Glutes' AND equipment = 'Bodyweight' AND difficulty_rank <= 2;
```

Should use `ix_ex_filter`. If it doesn't after you add columns to the view, the covering index no longer matches the predicate order — rebuild it rather than accepting a scan. At 3,000 rows a scan is survivable; the index matters because retrieval runs on every chat turn — including every degraded turn, where it is the *only* thing that runs.

Appendix C — Reference

C.1 Files at a glance

File	Produced by	Consumed by
out/meals.json	scrape_mealdb.py	build_catalog_db.py
flutter_assets/data/exercises.json	your backend export	build_catalog_db.py
flutter_assets/data/exercises.db	build_catalog_db.py	Flutter app, gateway
flutter_assets/data/meals.db	build_catalog_db.py	Flutter app, gateway
flutter_assets/data/vocab/*.json	build_catalog_db.py	gateway system prompt
flutter_assets/data/manifest.json	build_catalog_db.py	AssetInstaller, version handshake
flutter_assets/data/validation_report.json	build_catalog_db.py	you, before shipping
flutter_assets/data/normalized/*.json	build_catalog_db.py	review / re-import

C.2 Validation report — error kinds

Errors fail the build under `--strict`. Fix them **in the backend**, then rebuild; never patch the `.db` files by hand.

Kind	Meaning	Fix
exercise_without_name	Row has neither <code>exercise</code> nor <code>name</code>	Delete or name the row
exercise_duplicate_slug	Two exercises produce the same slug	Rename one; they're the same name modulo whitespace/case
exercise_without_muscle_group	No <code>target_muscle_group</code>	Assign one — retrieval can't reach it otherwise
muscle_name_unresolved	Prime/secondary/tertiary muscle isn't in the reference list	Add the muscle, or correct the spelling
muscle_group_unresolved	<code>target_muscle_group.name</code> isn't in <code>muscle_groups.json</code>	Same
difficulty_unresolved	Difficulty name isn't one of the eight	Correct it
legacy_muscle_id_ambiguous	One source id maps to several muscles	Give each muscle its own id in the backend
foreign_key_violation	A reference survived normalization but has no parent row	Report a bug in the builder
meal_without_id_or_name	Meal row unusable	Re-scrape

Kind	Meaning	Fix
<code>meal_duplicate_id</code>	Same <code>idMeal</code> twice	Deduplicate the scrape output

C.3 Validation report — warning kinds

Warnings don't fail the build, but each one is a small quality loss.

Kind	Meaning
<code>muscle_id_disagrees_with_name</code>	The row's id points at a different muscle than its name. Resolved by name; the backend row is mislabelled
<code>muscle_duplicate_merged</code>	Two rows differed only by whitespace and were merged
<code>muscle_group_duplicate_merged</code>	Same, for muscle groups (this is the <code>Trapezius</code> case)
<code>prime_mover_group_mismatch</code>	The prime mover doesn't belong to the stated muscle group
<code>muscle_has_no_group_mapping</code>	Muscle isn't in the builder's <code>MUSCLE_TO_GROUP</code> table — add it
<code>exercise_missing_demo_link</code>	No demonstration video; the detail screen will look empty
<code>meal_without_instructions</code>	Meal has no steps
<code>meal_slug_collision_suffixed</code>	Two meals produced the same slug; the second got its id appended
<code>difficulty_missing_from_source</code>	One of the eight canonical levels isn't in your file

C.4 Vocabulary counts (current data)

Vocabulary	Source rows	After cleanup
Muscle groups	17	16 (<code>Trapezius</code> merged)
Prime mover muscles	26	24 (whitespace duplicates merged)
Distinct muscle ids in source	17 for 26 rows	3 ids flagged ambiguous
Difficulty levels	8	8, re-ranked 1–8
Equipment, movement patterns, body regions	derived	derived from the exercise rows

C.5 Glossary

Candidate set — the ids retrieval returned for one turn. The model may cite nothing else.

Card — a row of `exercise_card` / `meal_card`. The unit of exchange between retrieval, the model, the gateway and the UI.

Degraded mode — serving retrieval results with a canned line when the model is unreachable, out of quota, or erroring. See §4.11.

Facet — a structured filter (muscle group, equipment, difficulty rank, movement pattern). What tool arguments compile into.

Fast path — skipping turn A when a regex classifier extracts facets confidently. On cloud, the primary cost lever. See §4.7.

Gateway — your server between the app and Ollama Cloud. Owns the prompt, retrieval, validation, the API key and the upstream concurrency budget.

GPU-time — Ollama Cloud's unit of metering. Not tokens, not requests. Driven by model weight class, generation length and prompt length.

Ollama Cloud — Ollama's hosted inference service, reached at `https://ollama.com/api/*` with a bearer API key. Same API as the local daemon, minus `num_ctx` and `keep_alive`.

Prime mover — the muscle primarily responsible for a movement. `role='prime'` in `exercise_muscle`.

Ref — an id the model returned in `exercise_refs` / `meal_refs`, before validation.

Relaxation ladder — the fixed order in which filters are dropped when a query returns nothing.

Slug — a URL-safe id derived from a name (`Bodyweight Glute Bridge` → `bodyweight-glute-bridge`).

Snapshot — denormalized card data the gateway attaches to each ref so an app with an older database can still render it.

Structured output — Ollama's `format` parameter constraining generation to a JSON Schema. Works identically on cloud models.

Thinking — gemma4's optional reasoning pass, returned in `message.thinking`. Disabled via `"think": false`. See §3.12.

Turn A / Turn B — the tool-calling round trip and the answer-writing round trip. Two upstream requests, two occupancies of your concurrency slot.

Vocabulary block — the ~500-token list of legal filter values embedded in the system prompt, generated from the database.

C.6 Pre-launch checklist

Data

- ☐ `validation_report.json` has zero errors; build runs with `--strict` in CI
- ☐ `name_ar` populated for meals and ingredients
- ☐ `ingredient.*_100g` filled from USDA FoodData Central
- ☐ TheMealDB supporter key in place (the `"1"` test key is not for public releases)
- ☐ Meal images mirrored to your own CDN; attribution visible in the app

Upstream — Ollama Cloud

- ☐ `OLLAMA_API_KEY` in a secret manager; separate keys per environment; absent from repo, images and logs
- ☐ Key rotation procedure written down and rehearsed once
- ☐ Deploy-time assertion: model available, tool calling works, `format` returns schema-valid JSON (§7.3)

- ☐ `think: false` on both turns; verified in a captured request, not just in the code
- ☐ `num_ctx` and `keep_alive` removed from all request bodies
- ☐ `OLLAMA_MAX_INFLIGHT` matches the plan actually being paid for, **divided by the replica count** (§4.6)
- ☐ `OLLAMA_QUEUE_DEPTH` set; worst-case wait at that depth calculated and accepted
- ☐ Plan sized against measured upstream-seconds-per-message × projected volume — **not** the Free tier
- ☐ Gateway region chosen from measured `ttft_ms`, not from intuition (§7.4)

Gateway

- ☐ JWT verification, per-user rate limit and monthly quota enforced
- ☐ Bounded queue with a `retryable` error response; `Busy` returns in under a second
- ☐ Retry policy correct: never on 401/403, never on 429, turn B only when zero tokens sent
- ☐ Degraded mode implemented and tested by revoking the key in staging (§4.11)
- ☐ Global daily circuit breaker wired to degraded mode
- ☐ Per-turn logging with `upstream_ms_*`, `queue_ms`, `upstream_status`, `ttft_ms`, `degraded`
- ☐ No code path calls `chat()` directly — everything goes through `handle_chat()` (§4.5)
- ☐ Fast-path classifier emits *vocabulary values*, generated from the database, not raw user words (§4.7)
- ☐ Alerts on `refs_dropped`, 429 rate, degraded rate and upstream seconds per 1k messages
- ☐ `/healthz` distinguishes down from degraded
- ☐ System prompt and tool defs in version control

App

- ☐ `AssetInstaller` verifies checksums and survives being killed mid-copy
- ☐ Databases opened `readOnly: true`
- ☐ FTS input escaped before `MATCH`
- ☐ Cards hydrate from the local database, snapshots only as fallback
- ☐ Snapshot cards render but don't navigate
- ☐ Degraded replies styled distinctly; no retry chip on `quota`
- ☐ Chat disabled offline with a clear notice; catalog still fully usable
- ☐ Composer text preserved on error
- ☐ `chat_card_tapped` analytics wired

Quality & policy

- ☐ 30-question eval set, half Arabic, run before every prompt change **and weekly on a schedule**
- ☐ Eval set includes the degraded case
- ☐ Safety cases return `safety_flag` and no exercise refs
- ☐ Safety classifier runs on the gateway, so it still works in degraded mode
- ☐ No URLs ever appear in `reply`
- ☐ Macros shown with an "estimated" badge everywhere they appear
- ☐ **Privacy policy discloses third-party AI processing and the regions involved (§7.7)**
- ☐ **Free-text health detail in the profile block reviewed for minimisation before it is sent upstream**